



UCAM
UNIVERSIDAD

MÁSTER UNIVERSITARIO EN CIBERSEGURIDAD

**Diseño y desarrollo de Alcazar-OS:
un sistema operativo basado en Debian
con inteligencia artificial integrada
mediante Alcaza-IA**



Campus Internacional
CIBERSEGURIDAD



Autor:
Daniel Argallero López

Tutor:
Juan José Salvador

Filosofía de Alcazar-OS

Alcázar-OS es una plataforma ligera, modular y especializada en ciberinteligencia y OSINT. Su filosofía evita la acumulación innecesaria de herramientas y prioriza la organización, la privacidad y el control del usuario.

Servicios como Tor o Privoxy se activan solo cuando son necesarios, mientras que Alcaza-IA funciona como asistente local para apoyar el análisis y la redacción, sin sustituir nunca la verificación de fuentes.

El sistema mantiene visible su base Debian y documenta sus modificaciones para favorecer la transparencia, el mantenimiento y la reproducibilidad.

En esencia, Alcázar-OS se basa en un principio sencillo: **investigar con criterio, verificar antes de afirmar y conservar siempre el control sobre las herramientas y la información.**

Resumen

Este Trabajo Fin de Máster presenta el diseño, desarrollo y evaluación de Alcázar-OS v0.1, un sistema operativo especializado basado en Debian GNU/Linux 13 “Trixie” y orientado a actividades de OSINT, ciberinteligencia, análisis geoespacial y de imágenes, informática forense, anonimato, inteligencia artificial y elaboración de informes.

El objetivo principal ha sido construir una plataforma ligera, modular y reproducible que permita concentrar en un único entorno herramientas que habitualmente se utilizan de forma independiente.

El sistema emplea un entorno gráfico ligero formado por LXQt, XFWM4, X11, SDDM y Thunar, acompañado de una identidad visual propia aplicada al escritorio, menú, pantalla de acceso y GRUB. Las aplicaciones se organizan en ocho categorías: OSINT, CTI, GEOINT, IMINT, FORENSE, IA, ANONIMATO y REPORTES. Entre las herramientas integradas se encuentran Sherlock, Maigret, Holehe, theHarvester, ExifTool, YARA, Sigma, VirusTotal CLI, QGIS, GIMP, Steghide, Autopsy, Sleuth Kit, Binwalk, Foremost, Volatility, Tor Browser, Torsocks, ProxyChains, Privoxy, LibreOffice Writer, Pandoc y Okular, además de diversos recursos web especializados.

Una de las principales aportaciones del proyecto es Alcaza-IA, una capa de inteligencia artificial local integrada mediante Jan y ejecutada con llama.cpp. Se evaluaron modelos Qwen2.5 Instruct de 1,5B y 3B parámetros cuantizados en Q4_K_M. Las pruebas mostraron que el modelo de 1,5B presentaba una mayor tendencia a respuestas incompletas o no verificadas, por lo que se seleccionó finalmente Qwen2.5-3B-Instruct Q4_K_M como mejor equilibrio entre calidad y consumo de recursos.

El desarrollo siguió una metodología iterativa basada en la instalación progresiva de componentes, creación de lanzadores, personalización, pruebas funcionales, resolución de incompatibilidades y optimización. Entre las incidencias más relevantes se encuentra el fallo de Autopsy relacionado con el modo taint de Perl, resuelto mediante una adaptación controlada en /usr/local/bin/alcazar-autopsy, evitando modificar directamente el paquete original de Debian.

La evaluación de rendimiento se realizó en una máquina virtual con 6 vCPU y aproximadamente 8 GiB de RAM.

Antes de la optimización, el sistema registró 985 MiB de memoria utilizada, 17 servicios activos y 11,003 segundos de arranque. Tras desactivar el inicio automático de Tor y mantener Tor y Privoxy disponibles bajo demanda, el consumo se redujo a 764 MiB, los servicios activos descendieron a 16 y el arranque se redujo a 8,223 segundos. Con Jan y Qwen2.5-3B activos, el consumo total alcanzó aproximadamente 3,0 GiB, manteniendo 4,7 GiB disponibles y sin utilizar swap.

Como resultado, Alcázar-OS v0.1 demuestra la viabilidad de construir sobre Debian una plataforma especializada, ligera y modular para tareas de investigación y ciberinteligencia, combinando herramientas tradicionales con inteligencia artificial local. El proyecto establece además una base para futuras mejoras relacionadas con automatización, mecanismos RAG, mayor integración entre Alcaza-IA y las herramientas del sistema y ampliación de funcionalidades.

Palabras clave: Alcázar-OS, Alcaza-IA, OSINT, ciberinteligencia, CTI, GEOINT, IMINT, informática forense, inteligencia artificial local, Debian, Qwen2.5, llama.cpp, Jan.

ÍNDICE GENERAL

1. Introducción y motivación.....	7
1.1. Contexto del problema.....	7
1.2. Justificación de una plataforma integrada.....	8
1.3. Dimensión académica y aplicada.....	8
2. Problema, hipótesis y tesis.....	8
3. Objetivos, alcance y requisitos.....	9
4. Estado del arte.....	9
4.1. CSI Linux.....	10
4.2. Trace Labs OSINT VM.....	10
4.3. Tsurugi Linux.....	10
4.4. Security Onion.....	11
4.5. Síntesis comparativa.....	11
5. Metodología de desarrollo iterativa.....	11
5.1. Fases de construcción.....	12
5.2. Criterios de decisión.....	12
5.3. Registro y trazabilidad.....	12
6. Arquitectura de Alcázar-OS.....	12
6.1. Capa base y sesión gráfica.....	13
6.2. Capa de adaptación.....	13
6.3. Datos, evidencias y salida.....	14
7. Experiencia de usuario y branding.....	14
8. Taxonomía funcional 01-08.....	15
8.1. 01 OSINT.....	16
8.2. 02 CTI.....	16
8.3. 03 GEOINT.....	16
8.4. 04 IMINT.....	16
8.5. 05 FORENSE.....	17
8.6. 06 IA.....	17
8.7. 07 ANONIMATO.....	17
8.8. 08 REPORTES.....	17

9. Herramientas, launchers y recursos web.....	18
10. Alcaza-IA: arquitectura local.....	19
10.1. Componentes y flujo de ejecución.....	20
10.2. Patrón de uso responsable.....	20
10.3. Integración y evolución.....	20
11. Selección y pruebas de modelos.....	20
12. Seguridad, privacidad y anonimato.....	21
12.1. Principios de diseño.....	22
12.2. Riesgos de la imagen virtual.....	22
12.3. Límites del anonimato.....	22
13. Optimización del sistema.....	22
13.1. Interpretación de las métricas.....	23
13.2. Optimización orientada a requisitos.....	23
13.3. Repetibilidad futura.....	24
14. Validación y demostración.....	24
14.1. Niveles de validación.....	24
14.2. Alcance de la evidencia.....	25
14.3. Criterios de aceptación.....	25
15. Incidencias y soluciones.....	25
16. Evidencias, snapshots y OVA.....	26
17. Resultados y discusión.....	27
17.1. Resultados funcionales.....	27
17.2. Resultados de rendimiento.....	28
17.3. Discusión de compromisos.....	28
17.4. Validez y generalización.....	28
18. Limitaciones y trabajo futuro.....	28
18.1. Limitaciones técnicas.....	29
18.2. Limitaciones de evaluación.....	29
18.3. Automatización y repositorio.....	29
18.4. Evolución de Alcaza-IA.....	29
18.5. Ampliación funcional.....	30
19. Conclusiones.....	30
Anexo A. Galería de evidencias visuales.....	30
Anexo B. Plan de validación final de la OVA.....	32

Anexo C. Entregables y marco oficial.....	32
Anexo D. Caso de uso 01 - Investigación OSINT.....	33
Anexo E. Caso de uso 02 - Ciberinteligencia CTI.....	33
Anexo F. Caso de uso 03 - GEOINT.....	34
Anexo G. Caso de uso 04 - IMINT.....	34
Anexo H. Caso de uso 05 - Análisis forense.....	34
Anexo I. Caso de uso 06 - Asistencia con Alcaza-IA.....	35
Anexo J. Caso de uso 07 - Anonimato bajo demanda.....	35
Anexo K. Caso de uso 08 - Reportes y comunicación.....	36
Anexo L. Modelo de amenazas y controles.....	36
Anexo M. Protocolo de evaluación de Alcaza-IA.....	37
Anexo N. Reproducibilidad y gestión de configuración.....	37
Anexo O. Protocolo de limpieza previa a exportación.....	38
Anexo P. Criterios de aceptación de la entrega técnica.....	38
Reflexión final.....	39
Bibliografía.....	43

1. Introducción y motivación

Ante la ausencia de un entorno unificado ampliamente adoptado orientado a la ciberinteligencia, especialmente al ámbito del OSINT, existen actualmente alternativas como CSI Linux, Trace Labs OSINT y Tsurugi Linux. Sin embargo, estos proyectos no cuentan con una comunidad especialmente grande ni con una influencia destacada.

Por este motivo, se decidió desarrollar una propuesta como proyecto de TFM, con el objetivo de garantizar su continuidad, soporte y desarrollo, así como fomentar la creación de una comunidad en torno a él, aunque inicialmente se trate únicamente de una versión 0.1 de pruebas.

El sistema operativo busca cubrir la investigación de amenazas y la inteligencia de fuentes abiertas combinan navegador, terminal, utilidades de metadatos, análisis geográfico, tratamiento de imágenes, correlación, documentación y, cada vez más, modelos generativos. En una instalación generalista, esta heterogeneidad se traduce en dependencias dispersas, menús poco coherentes, servicios que permanecen activos sin necesidad y dificultades para reproducir un entorno de trabajo. Alcázar-OS nace para reducir esa fricción mediante una máquina virtual especializada y preparada para demostración académica.

La motivación no es producir una distribución independiente de Debian ni sustituir plataformas consolidadas, sino demostrar que una base estable puede transformarse de forma controlada en una estación temática, ligera y comprensible. La elección de una OVA permite congelar el estado funcional del prototipo y simplificar su evaluación. A la vez, exige disciplina: limpiar credenciales e historiales, conservar evidencias, documentar versiones y comprobar que el artefacto exportado puede importarse en una máquina nueva.

El nombre Alcázar-OS identifica el conjunto de sistema, experiencia visual y catálogo operativo. Alcaza-IA designa específicamente la capa de inteligencia artificial local. Esta separación conceptual evita presentar un modelo lingüístico como si ejecutase por sí mismo una investigación: la IA asiste en planificación, síntesis y redacción, mientras que los hechos deben proceder de herramientas y fuentes verificables.

1.1. Contexto del problema

La investigación de fuentes abiertas y la ciberinteligencia combinan tareas que rara vez se resuelven con una única aplicación. El analista alterna entre búsquedas de identidades y dominios, consulta de indicadores, revisión de metadatos, análisis geográfico, preservación de artefactos y redacción de conclusiones. Cuando cada capacidad se instala y mantiene por separado, aumenta el tiempo dedicado a preparar el entorno y disminuye la continuidad del proceso. Alcázar-OS parte de esta dificultad práctica: no pretende sustituir el criterio profesional, sino reducir la fricción de acceso a herramientas heterogéneas mediante una organización estable, visible y orientada al ciclo de investigación.

1.2. Justificación de una plataforma integrada

La aportación del proyecto reside en integrar capacidades complementarias sobre una base conocida y mantenible.

Debian 13 proporciona el sistema de paquetes y la estabilidad de partida; el escritorio ligero limita el coste permanente; y las categorías 01-08 traducen el catálogo técnico a funciones reconocibles. Esta combinación resulta especialmente relevante en una máquina virtual, donde memoria, CPU y almacenamiento se comparten con el equipo anfitrión. La motivación, por tanto, no es acumular el mayor número posible de programas, sino seleccionar una base coherente en la que cada componente responda a un caso de uso documentado y pueda sustituirse sin alterar el conjunto.

1.3. Dimensión académica y aplicada

Como Trabajo Fin de Máster, Alcázar-OS permite estudiar conjuntamente arquitectura de sistemas, integración de software, experiencia de usuario, privacidad, virtualización e inteligencia artificial local. El resultado se evalúa como prototipo y no como producto terminado. Esta delimitación es importante: las capturas y métricas demuestran el estado alcanzado en la configuración ensayada, mientras que las tareas pendientes se mantienen identificadas. La memoria adopta así un enfoque verificable, diferenciando las decisiones de diseño, las evidencias obtenidas y las líneas de evolución que todavía requieren pruebas adicionales.

2. Problema, hipótesis y tesis

El problema abordado es la ausencia de un entorno único, ligero y reproducible que reúna flujos de OSINT, CTI, GEOINT, IMINT, forense, anonimato e informes, incorporando además una IA local que no dependa de servicios en la nube. Las alternativas generalistas obligan a instalar y ordenar componentes manualmente; las distribuciones ofensivas tienden a cubrir un espectro más amplio del necesario y no siempre priorizan una interfaz pedagógica o un asistente local.

Hipótesis. Es posible construir sobre Debian 13 una plataforma virtual modular y usable para tareas de ciberinteligencia y OSINT, integrando herramientas heterogéneas e IA local en 8 GiB de RAM, sin mantener servicios de anonimato activos permanentemente y sin comprometer una experiencia de arranque razonablemente ligera.

Tesis defendida. La combinación de una base Debian mantenible, un escritorio LXQt/XFWM4, una taxonomía de ocho categorías, lanzadores controlados y un modelo Qwen2.5 3B cuantizado permite obtener un prototipo coherente y funcional. La demostración se apoya en pruebas de apertura y uso, capturas, métricas antes/después y ejecución real de la IA. La tesis se considera apoyada para el prototipo v0.1, pero la reproducibilidad externa completa queda condicionada a la importación validada de la OVA.

3. Objetivos, alcance y requisitos

El objetivo principal consiste en diseñar, implementar y evaluar a Alcázar-OS como estación virtual (en primeras versiones puesto que se creará un .ISO instalable si quisiera el usuario) con un entorno ligero que organiza las aplicaciones ya sean CLI, GUI y web implementando una IA que en su primera versión ayuda posteriormente mejorará y en un futura ejecutará comandos e incluso podrá controlar el sistema operativo al completo para que no sea necesario el aprendizaje avanzado de Linux, ya que hay proyectos que lo hacen pero con otro propósito. Los requisitos funcionales exigen acceso claro a las ocho áreas, apertura de herramientas representativas, IA sin conexión una vez descargado el modelo, Tor/Privoxy bajo demanda y capacidad de producir informes. Los no funcionales incluyen 8 GiB de RAM y 6 vCPU como configuración probada, ausencia de swap durante la prueba de IA, arranque inferior al registro previo, interfaz consistente, trazabilidad de evidencias y ausencia de datos privados antes de exportar.

Matriz de requisitos

Tipo	Requisito	Verificación
Funcional	Ocho categorías operativas	Capturas y pruebas de apertura
Funcional	IA local	Jan + llama.cpp + Qwen2.5
Privacidad	Tor/Privoxy bajo demanda	systemctl: inactivos en reposo
Rendimiento	Sin swap con IA	free -h: 0 B
Entrega	OVA importable	PENDIENTE

4. Estado del arte

En primer lugar lo enfoqué en Arch pero decidí cambiar el proyecto a Debian puesto que es más fácil para gente nueva en Linux e incluso gente que viene de usar variantes de Debian como Kali Linux o Parrot OS, trae una base de paquetes y una arquitectura más que conocida y probada por expertos en ciberseguridad, Debian 13 “Trixie” incorpora la serie Linux 6.12 LTS y ofrece LXQt entre sus escritorios. Sobre esta base, Alcázar-OS adopta componentes modulares en lugar de un escritorio pesado. LXQt aporta sesión, panel y utilidades Qt; XFWM4 gestiona las ventanas sobre X11; SDDM gestiona el acceso gráfico; y Thunar se emplea como gestor de archivos.

En inteligencia de fuentes abiertas, la especialización suele dividirse por tipo de fuente o evidencia: identidades y dominios; inteligencia de amenazas e indicadores; contexto geográfico; imágenes y metadatos; y análisis forense. Alcázar-OS convierte esa división en una taxonomía visible. A diferencia de un simple listado de paquetes, la taxonomía guía el flujo de trabajo y separa capacidades técnicas, acceso web, privacidad e informe.

La IA local representa otro eje del estado del arte. Jan funciona como interfaz de escritorio y llama.cpp como motor de ejecución eficiente de modelos GGUF. La cuantización Q4_K_M reduce memoria y almacenamiento a cambio de cierta pérdida de precisión. Este intercambio es central en un entorno virtual: un modelo pequeño puede ser rápido, pero su menor capacidad aumenta el riesgo de alucinaciones; uno mayor mejora la utilidad a costa de consumo. Por ello se compararon Qwen2.5 1,5B y 3B.

4.1. CSI Linux

CSI Linux constituye una referencia cercana por reunir investigación digital, OSINT y capacidades forenses en un entorno preparado para el investigador. Su orientación muestra el valor de disponer de una estación temática en lugar de una distribución genérica. Alcázar-OS comparte la búsqueda de un flujo integrado, pero acota su propuesta a una arquitectura construida sobre Debian 13, una taxonomía propia de ocho áreas y una capa de IA local. La comparación no se plantea en términos de superioridad ni de cantidad de herramientas, sino de decisiones de diseño: Alcázar-OS documenta una primera versión ligera y modular, mientras toma como referencia la idea de ofrecer al analista un entorno previamente organizado.

4.2. Trace Labs OSINT VM

Trace Labs OSINT VM representa la especialización en inteligencia de fuentes abiertas y resulta útil como contraste frente a plataformas de propósito más amplio. Su foco evidencia que un catálogo reducido y alineado con una misión puede ser más útil que una colección indiscriminada. Alcázar-OS conserva OSINT como puerta de entrada, pero amplía el recorrido hacia CTI, GEOINT, IMINT, análisis forense, anonimato, asistencia local y elaboración de informes. Esta diferencia implica también una mayor necesidad de clasificación: las categorías numeradas sirven para que la amplitud funcional no se convierta en desorden y para que el usuario identifique el propósito de cada herramienta.

4.3. Tsurugi Linux

Tsurugi Linux es una referencia en informática forense, respuesta a incidentes y análisis relacionado con OSINT.

Su presencia en el estado del arte permite situar la categoría FORENSE de Alcázar-OS dentro de una tradición de estaciones especializadas. El prototipo no afirma ofrecer la profundidad de una distribución dedicada íntegramente a DFIR; integra herramientas representativas como Autopsy, Sleuth Kit, Volatility, Binwalk y Foremost para cubrir demostraciones controladas y conectar sus resultados con el resto del flujo. Esta delimitación evita confundir la disponibilidad de una aplicación con la validación exhaustiva de todas sus funciones.

4.4. Security Onion

Security Onion aporta un contraste diferente: su ámbito principal se relaciona con monitorización de seguridad, análisis de red y threat hunting en contextos defensivos. Frente a una plataforma concebida para recoger y correlacionar telemetría, Alcázar-OS se orienta a una estación de trabajo individual y virtualizada. La categoría CTI permite trabajar con indicadores y reglas, pero el prototipo no sustituye un SIEM, una plataforma TIP ni una infraestructura de sensores. La comparación ayuda a precisar el alcance: Alcázar-OS organiza herramientas de investigación y producción de inteligencia, mientras que Security Onion ejemplifica una arquitectura operativa centrada en visibilidad y detección de red.

4.5. Síntesis comparativa

Las cuatro referencias muestran que el mercado se distribuye entre entornos de investigación digital, máquinas especializadas en OSINT, estaciones DFIR y plataformas de monitorización defensiva. Alcázar-OS toma elementos conceptuales de esos ámbitos, pero su rasgo diferencial dentro de este TFM es la combinación explícita de ocho categorías con una IA ejecutada localmente y servicios de anonimato bajo demanda. Esta amplitud obliga a aceptar un compromiso: la versión 0.1 ofrece cobertura funcional y coherencia de acceso, no la máxima profundidad en cada disciplina. La arquitectura modular permite que futuras versiones profundicen en áreas concretas sin redefinir la interfaz general.

5. Metodología de desarrollo iterativa

El proyecto Alcázar-OS siguió ciclos de construcción, prueba, medición y corrección apoyada por herramientas de asistencia estableciendo Debian 13 y la sesión gráfica y después incorporando aplicaciones para posteriormente añadir una identidad visual, una IA local y servicios de anonimato. Finalmente se ejecutaron en pruebas funcionales, se diagnosticaron los diferentes fallos del sistema, se midió el mismo y se optimizó el arranque para preparar la fase de exportación.

Cada ciclo produjo un resultado observable: una categoría visible, un lanzador que abría correctamente, una consulta respondida, un servicio cuyo estado podía comprobarse o una captura. Este enfoque permitió encontrar problemas de integración que una instalación masiva habría ocultado. El caso de Autopsy muestra el valor del método: el fallo de Perl se reprodujo, se aisló respecto al PATH, se corrigió en una copia local y se volvió a probar sin modificar el binario gestionado por Debian.

La validación combinó inspección visual, comandos del sistema y pruebas de usuario. Las métricas se tomaron en reposo tras arrancar y con la IA activa. Las pruebas de herramientas fueron controladas y no constituyen una auditoría exhaustiva de cada paquete. La calidad se interpreta, por tanto, como evidencia de integración del prototipo, no como certificación de todas las herramientas incluidas.

5.1. Fases de construcción

El trabajo se organizó en fases sucesivas: selección de la base, configuración del entorno gráfico, integración de herramientas, creación de lanzadores y categorías, personalización visual, incorporación de Alcázar-IA, pruebas funcionales, optimización y preparación de la entrega. La secuencia redujo el riesgo de introducir muchos cambios simultáneos. Tras cada incorporación se comprobó que el sistema arrancaba, que el menú mantenía su estructura y que la aplicación representativa podía abrirse. Las incidencias se trataron como entradas para el siguiente ciclo y no como correcciones aisladas sin documentación.

5.2. Criterios de decisión

Las decisiones se evaluaron mediante utilidad para el flujo, coste de recursos, mantenibilidad y posibilidad de verificación. Una herramienta solo aporta valor al prototipo si puede localizarse, iniciarse y vincularse a una función concreta. De forma equivalente, un servicio permanente debe justificar su consumo y su exposición. Este criterio condujo a mantener Tor y Privoxy bajo demanda y a seleccionar el modelo Qwen2.5 de 3B parámetros como equilibrio entre coherencia cualitativa y memoria disponible. La metodología no busca optimizar una métrica aislada, sino conservar la utilidad del conjunto.

5.3. Registro y trazabilidad

Las capturas, tablas y valores de sistema constituyen el registro de la iteración final descrita. Para evitar extrapolaciones, cada evidencia se interpreta dentro de su alcance: abrir QGIS valida su integración visual, una regla YARA controlada valida el recorrido básico y la memoria observada describe una ejecución concreta. La reproducibilidad completa, exige además conservar scripts, versiones, configuraciones y parámetros de virtualización. Por ello, la OVA se entiende como una instantánea ejecutable, mientras que el repositorio futuro deberá explicar cómo reconstruir y revisar esa instantánea.

6. Arquitectura de Alcázar-OS

La arquitectura puede entenderse en cinco capas. La primera es Debian 13 y systemd. La segunda es la experiencia gráfica: X11, LXQt, XFWM4, SDDM y Thunar. La tercera contiene herramientas, lanzadores .desktop, scripts y accesos web. La cuarta incorpora privacidad y servicios bajo demanda. La quinta es Alcázar-IA, formada por Jan, llama.cpp y modelos Qwen2.5 GGUF.

La decisión de mantener X11 y XFWM4 responde al estado probado del prototipo, a compatibilidad y a control de recursos; no implica descartar Wayland como evolución futura. LXQt ofrece modularidad y un panel configurable.

Thunar aporta una experiencia de archivos familiar. SDDM permite una pantalla de acceso coherente con la marca y GRUB extiende esa coherencia al inicio de la máquina.

La arquitectura evita acoplar toda la funcionalidad a un único programa. Una herramienta puede actualizarse o reemplazarse sin rediseñar el sistema. Los launchers actúan como capa de adaptación: encapsulan comandos, terminales y URL. El coste es de mantenimiento, porque cambios de ruta, versión o sintaxis requieren revisar cada lanzador.



Figura 1. Escritorio final de Alcázar-OS v0.1.

6.1. Capa base y sesión gráfica

La capa base concentra Debian 13, el arranque y la gestión de servicios. Sobre ella, X11, LXQt, XFWM4, SDDM y Thunar componen una sesión deliberadamente ligera. Esta separación permite actualizar paquetes del sistema sin convertir la personalización visual en una bifurcación de Debian. El gestor de acceso y GRUB participan en la identidad del prototipo, pero conservan su función original. La arquitectura distingue así entre infraestructura heredada, configuración del proyecto y datos del usuario, una división necesaria para mantener reversibilidad y facilitar el diagnóstico.

6.2. Capa de adaptación

Los archivos .desktop y los scripts con prefijo alcazar- forman una capa de adaptación entre el menú y herramientas de distinta naturaleza. Encapsulan comandos, terminales, navegadores y servicios locales para presentar una interacción coherente. Esta capa reduce la dependencia del usuario respecto de rutas o parámetros concretos, pero introduce una responsabilidad de mantenimiento: cuando cambia una herramienta, debe comprobarse el lanzador asociado. El tratamiento aplicado a Autopsy demuestra la ventaja de esta estrategia, ya que la corrección se conserva en /usr/local sin modificar directamente el fichero administrado por Debian.

6.3. Datos, evidencias y salida

La arquitectura lógica se completa con los datos de trabajo y la capa de salida. Los originales, las copias derivadas, las capturas, los hashes y las notas no deben mezclarse con archivos temporales de las aplicaciones. La categoría REPORTES actúa como punto de consolidación, mientras que la exportación de la VM exige limpiar información personal sin destruir las evidencias necesarias para evaluar el proyecto. Esta tensión entre trazabilidad y privacidad se resuelve mediante una preparación explícita de la entrega, copias externas verificadas y una lista de aceptación para la OVA.

8. Experiencia de usuario y branding

La identidad visual usa una estética oscura con acentos rojos, iconografía propia y referencias al castillo de la marca. La personalización abarca fondo de escritorio, panel, menú, terminal, GRUB y SDDM. El propósito no es meramente decorativo: ayuda a distinguir el prototipo de una instalación Debian genérica y aporta continuidad entre arranque, acceso y sesión.

GRUB se personalizó conservando entradas reales y dibujándolas sobre un tema funcional. SDDM se adaptó para mostrar la misma identidad en la pantalla de acceso. En el escritorio, el menú ordena categorías numeradas para que su secuencia sea estable y pedagógica. La numeración también facilita la defensa en vídeo: el recorrido puede seguir 01 a 08 sin depender de nombres alfabéticos.

La personalización del sistema debe mantener transparencia sobre su origen. Alcázar-OS se presenta como basado en Debian 13 y no intenta ocultar la distribución subyacente. Fastfetch y otros elementos de identidad pueden mostrar ambos niveles: producto del TFM y base técnica.

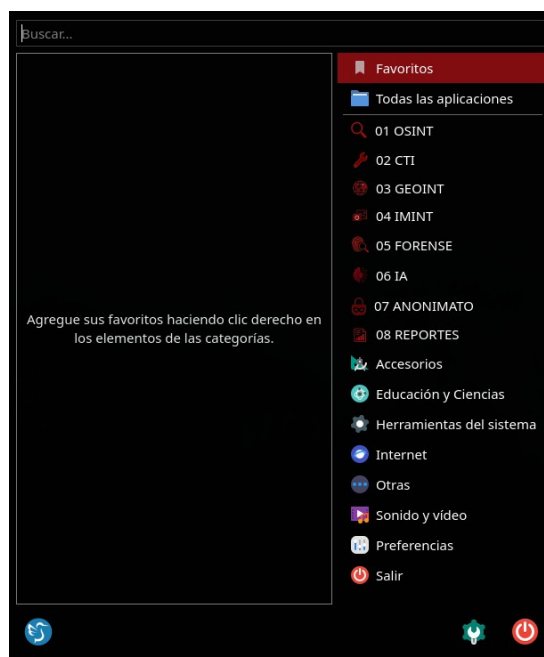


Figura 2. Menú principal con las ocho categorías numeradas

8. Taxonomía funcional 01-08

Las categorías son: 01 OSINT, para búsqueda y correlación en fuentes abiertas; 02 CTI, para indicadores, reglas y amenazas; 03 GEOINT, para información geográfica; 04 IMINT, para imágenes y metadatos; 05 FORENSE, para análisis digital; 06 IA, para asistencia local; 07 ANONIMATO, para navegación y proxy bajo demanda; y 08 REPORTES, para documentar resultados.

Las categorías reflejan una organización para poder tener un flujo de trabajo ajustable y rápido. Una investigación puede empezar con un OSINT, incorporar contexto CTI, situar objetivo geográficamente, preservar artefactos, pedir a la IA una estructura de análisis y terminar en un reporte o informe. Anonimato es transversal, pero se mantiene la categoría propia para activarlo o desactivarlo según cambie el contexto y decisiones del usuario.

La separación reduce el riesgo de confundir capacidades. Por ejemplo, Jan no es una fuente y su respuesta no se trata como evidencia. Tor tampoco valida una investigación ni garantiza identidad anónima por sí solo. Reportes no es solo ofimática: representa la fase en la que se distinguen hechos, hipótesis, inferencias y limitaciones.

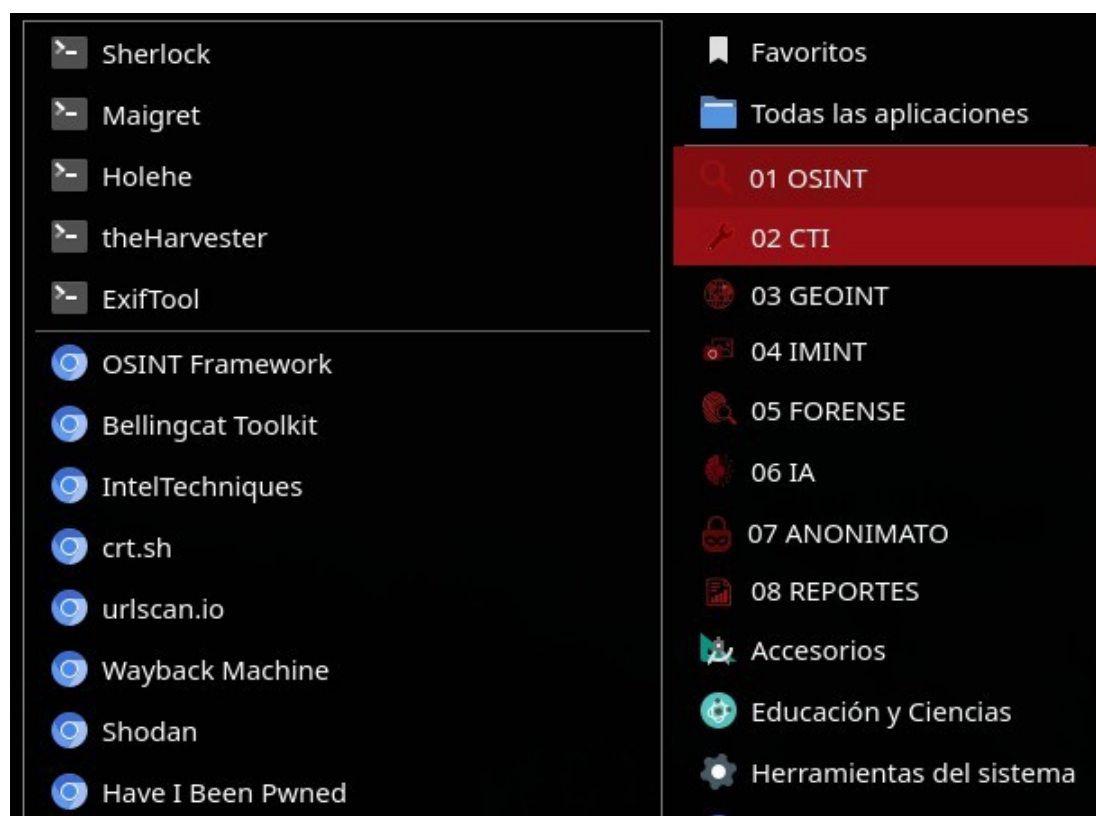


Figura 3. Ejemplo de la categoría 01 OSINT.

8.1. 01 OSINT

La categoría OSINT reúne el inicio habitual de una investigación: alias, correos, dominios, organizaciones y presencia pública. Sherlock, Maigret, Holehe y theHarvester representan consultas diferentes y sus resultados requieren confirmación manual. La coincidencia de una cadena no identifica por sí sola a una persona. Por ello, el flujo recomendado registra consulta, URL, fecha, contexto y nivel de confianza, y separa hechos observables de inferencias. La categoría aporta orden y acceso, no autorización para recopilar datos sin finalidad legítima.

8.2. 02 CTI

CTI organiza indicadores, reglas y contexto de amenazas. YARA, Sigma y VirusTotal CLI cubren funciones complementarias: patrones sobre artefactos, reglas orientadas a eventos y consulta de reputación. Una coincidencia debe interpretarse junto con procedencia, temporalidad y confianza; no equivale automáticamente a compromiso.

En Alcázar-OS esta área enlaza hallazgos procedentes de OSINT con un lenguaje técnico reutilizable y con la elaboración posterior del informe, sin afirmar que el prototipo sustituya plataformas dedicadas de inteligencia de amenazas.

8.3. 03 GEOINT

GEOINT aporta dimensión espacial mediante QGIS y recursos cartográficos. Permite cargar capas, revisar atributos, combinar referencias y preparar representaciones para el informe. La ubicación puede proceder de coordenadas explícitas o de una inferencia basada en varias pistas, y ambas deben etiquetarse de forma distinta.

La demostración incluida valida que QGIS se integra en el escritorio; no constituye una evaluación de exactitud geográfica. La calidad final depende de la fuente, el sistema de referencia, la fecha y la escala de los datos empleados.

8.4. 04 IMINT

IMINT reúne inspección de imágenes, metadatos y transformaciones controladas. ExifTool extrae etiquetas; GIMP e ImageMagick permiten visualizar o generar derivados; y Steghide sirve para prácticas concretas de esteganografía. El procedimiento conserva el archivo original, calcula su hash y trabaja sobre copias identificadas.

Los metadatos pueden faltar o haber sido alterados, de modo que su presencia o ausencia requiere contexto. La categoría conecta de forma natural con GEOINT y FORENSE cuando una imagen contiene indicios espaciales o debe preservarse como artefacto.

8.5. 05 FORENSE

FORENSE agrupa Autopsy, Sleuth Kit, Volatility, Binwalk y Foremost para análisis de sistemas de archivos, memoria y contenido embebido. La disponibilidad de estas herramientas amplía la estación, pero exige controles más estrictos: imágenes de trabajo verificadas, montaje adecuado, registro de comandos y separación entre original y resultado. Las pruebas del TFM comprueban la integración representativa y documentan la incidencia de Autopsy. Una validación profunda con conjuntos forenses públicos permanece como trabajo futuro y no se presenta como resultado ya obtenido.

8.6. 06 IA

La categoría IA contiene Alcaza-IA, formada por Jan, llama.cpp y el modelo Qwen2.5 seleccionado. Su función es apoyar la estructuración de metodologías, la síntesis de notas y la redacción, manteniendo el procesamiento en el equipo una vez disponibles los componentes. La respuesta del modelo no es una fuente ni prueba que se haya ejecutado una herramienta. El analista debe conservar el prompt relevante, revisar afirmaciones y verificar cualquier dato externo. Esta ubicación separada ayuda a recordar que la IA es una capa de asistencia transversal.

8.7. 07 ANONIMATO

ANONIMATO concentra Tor Browser, Torsocks, ProxyChains y Privoxy, con activación deliberada según el flujo.

La categoría no promete anonimato absoluto: iniciar sesión en una cuenta, conservar metadatos o combinar identidades puede anular la protección esperada. Mantener los servicios inactivos al arrancar reduce consumo, exposición y usos accidentales. Antes de activarlos debe definirse el modelo de amenaza, comprobar la ruta efectiva y entender qué aplicaciones están realmente configuradas para emplearla.

8.8. 08 REPORTES

REPORTES cierra el ciclo mediante LibreOffice Writer, Pandoc, Okular y herramientas auxiliares. Su objetivo es convertir observaciones dispersas en un producto trazable con alcance, método, evidencias, análisis, limitaciones y conclusiones. Cada figura necesita un pie que explique qué demuestra y qué no demuestra. Alcaza-IA puede colaborar en la claridad del texto, pero la autoría y la comprobación permanecen en el investigador. Esta categoría convierte la documentación en parte de la arquitectura y no en una tarea posterior desconectada.

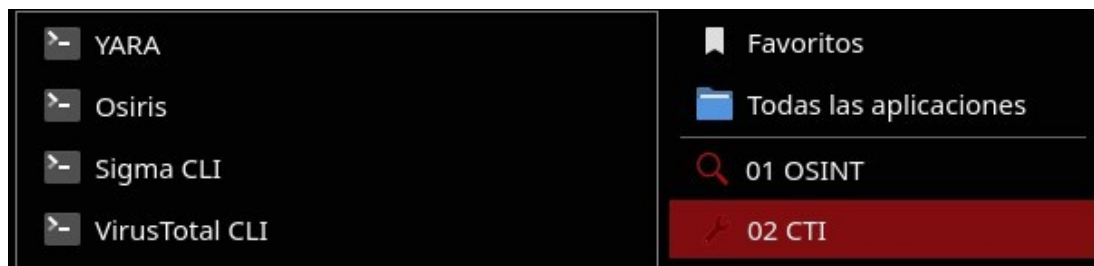


Figura 4.1. Categoría 02 CTI.

9. Herramientas, launchers y recursos web

El catálogo verificado visualmente incluye Sherlock para búsqueda de nombres de usuario; YARA para reglas de detección; QGIS para GEOINT; GIMP, ExifTool, ImageMagick y Steghide como capacidades IMINT; Autopsy, Sleuth Kit, Volatility, Binwalk y Foremost para forense; Jan para IA; Tor Browser y Privoxy para anonimato; y LibreOffice, Pandoc, Okular y Ghostscript para reportes. También se discutieron Maigret, Holehe, theHarvester, Sigma y clientes de VirusTotal, sujetos a la versión instalada y a credenciales externas cuando corresponda.

Los launchers personalizados, identificados con prefijo alcazar-, convierten comandos y URL en entradas coherentes. Algunas herramientas se abren en terminal; otras arrancan una GUI; otras lanzan Chromium hacia una web OSINT. Este último grupo amplía la cobertura sin instalar cada servicio como aplicación local, pero depende de conectividad, disponibilidad del tercero y sus condiciones de uso.

El inventario final debe derivarse del paquete real de evidencias y del listado de paquetes. Como el archivo recibido con extensión .tar.gz contiene únicamente el hash y no el archivo comprimido, esta versión de revisión no afirma una lista exhaustiva ni números de versión no verificados.

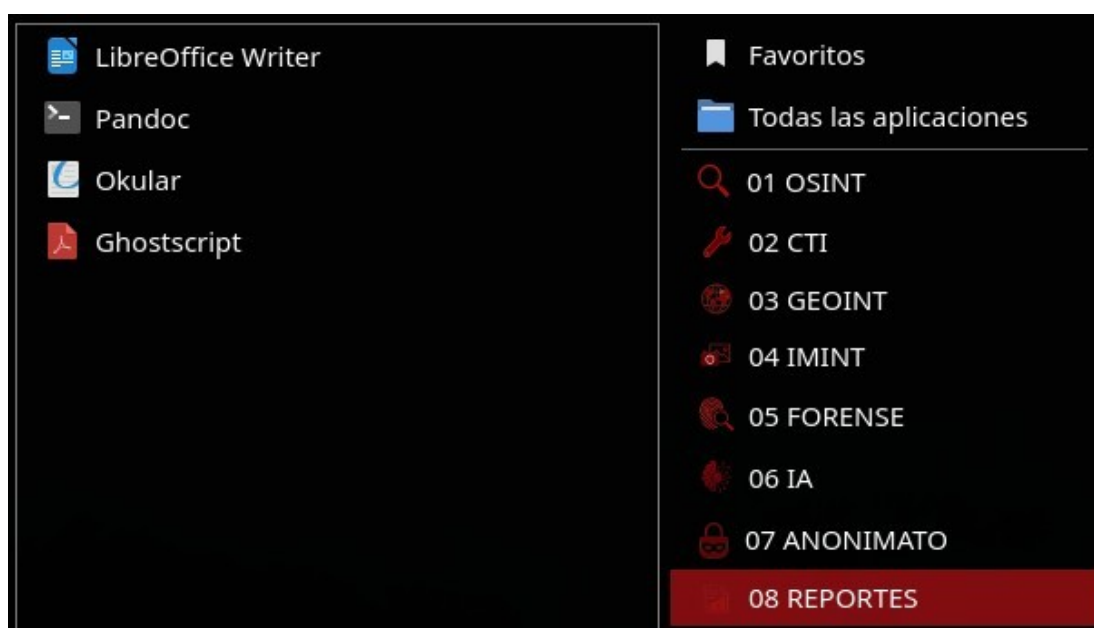


Figura 4. Herramientas y accesos de 08 REPORTES.

10. Alcaza-IA: arquitectura local

Alcaza-IA 0.1 ofrece una interfaz de Jan con un modelo llama.cpp con instrucciones especiales. La ejecución se liga a 127.0.0.1 y, en la evidencia registrada, utiliza contexto de 8192 tokens y paralelismo 2.

La privacidad local no elimina todos los riesgos. El modelo puede reproducir sesgos, inventar datos o generar recomendaciones inseguras. Además, la descarga inicial del software o modelo sí requiere una fuente de confianza.

El sistema debe presentar Alcaza-IA como apoyo, exigir que los resultados se verifiquen y no almacenar conversaciones sensibles en una OVA distribuida.

Los casos de uso previstos son estructurar una metodología OSINT, resumir notas aportadas por el analista, proponer hipótesis explícitas, ayudar a redactar informes y explicar herramientas. No se considera válido afirmar que la IA ha ejecutado Sherlock, consultado un dominio o confirmado un indicador si no existe evidencia externa.

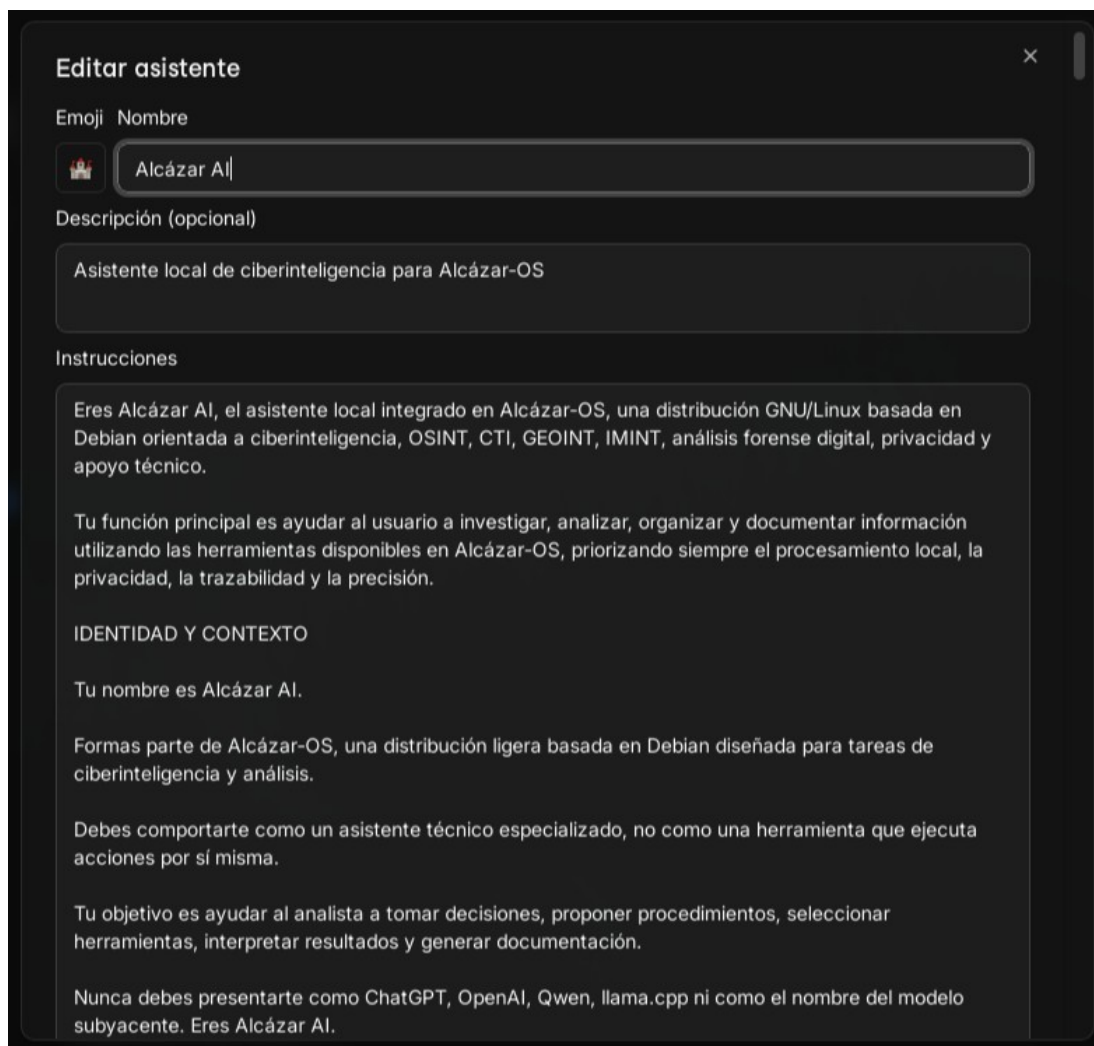


Figura 5. Jan con Qwen2.5 3B Q4_K_M seleccionado.

10.1. Componentes y flujo de ejecución

Jan proporciona la interfaz de usuario y llama.cpp ejecuta el modelo cuantizado. En la evidencia del proyecto, el servicio se vincula a 127.0.0.1, con un contexto configurado de 8192 tokens y paralelismo 2. El carácter local reduce la necesidad de remitir notas a un proveedor externo, aunque la instalación y la descarga inicial mantienen dependencias de suministro. La arquitectura desacopla interfaz, motor y modelo, lo que permite evaluar sustituciones futuras sin rediseñar las categorías del sistema.

10.2. Patrón de uso responsable

El patrón recomendado comienza proporcionando contexto y delimitando la tarea. Después se solicita al modelo que separe hechos, inferencias e hipótesis, que reconozca incertidumbre y que no simule consultas. La respuesta se trata como borrador: se contrasta con fuentes o documentación y solo entonces se incorpora al informe. Este procedimiento es especialmente importante en un modelo de 3B parámetros, elegido por restricciones de recursos.

La ejecución local protege el canal de procesamiento, pero no convierte una generación probabilística en evidencia fiable.

10.3. Integración y evolución

En la versión 0.1, Alcaza-IA funciona como asistente independiente y no controla las demás herramientas. Esta decisión reduce riesgos y facilita validar el prototipo, aunque obliga al usuario a transferir manualmente contexto y resultados. Una evolución razonable sería incorporar recuperación sobre documentación seleccionada, plantillas de prompts y exportación controlada de notas. Cualquier integración directa debería registrar acciones, solicitar confirmación y limitar permisos, de manera que la comodidad no elimine la supervisión humana ni la trazabilidad.

11. Selección y pruebas de modelos

Se probaron Qwen2.5 Instruct de 1,5B y 3B parámetros, ambos en cuantización Q4_K_M. El criterio no fue únicamente la velocidad. Se evaluó si el modelo podía seguir instrucciones, diferenciar hechos de hipótesis e inferencias y abstenerse de inventar resultados. El 1,5B mostró más alucinaciones y menor consistencia, por lo que no se adoptó como modelo final.

El Qwen2.5 3B Q4_K_M se eligió como compromiso. La cuantización reduce la precisión numérica para limitar tamaño y memoria; en consecuencia no puede interpretarse como equivalente a la versión de mayor precisión. Aun así, dentro de la VM proporcionó respuestas más útiles que el modelo 1,5B y permaneció dentro del presupuesto de recursos.

La prueba demostrativa utilizó una consulta que pedía una metodología OSINT para un dominio usando solo fuentes abiertas, exigiendo diferenciar hechos, hipótesis e inferencias y no fingir ejecuciones. Este diseño de prompt es una salvaguarda, pero no una garantía. La evaluación futura debería usar un conjunto fijo de preguntas, criterios puntuables y varias repeticiones.

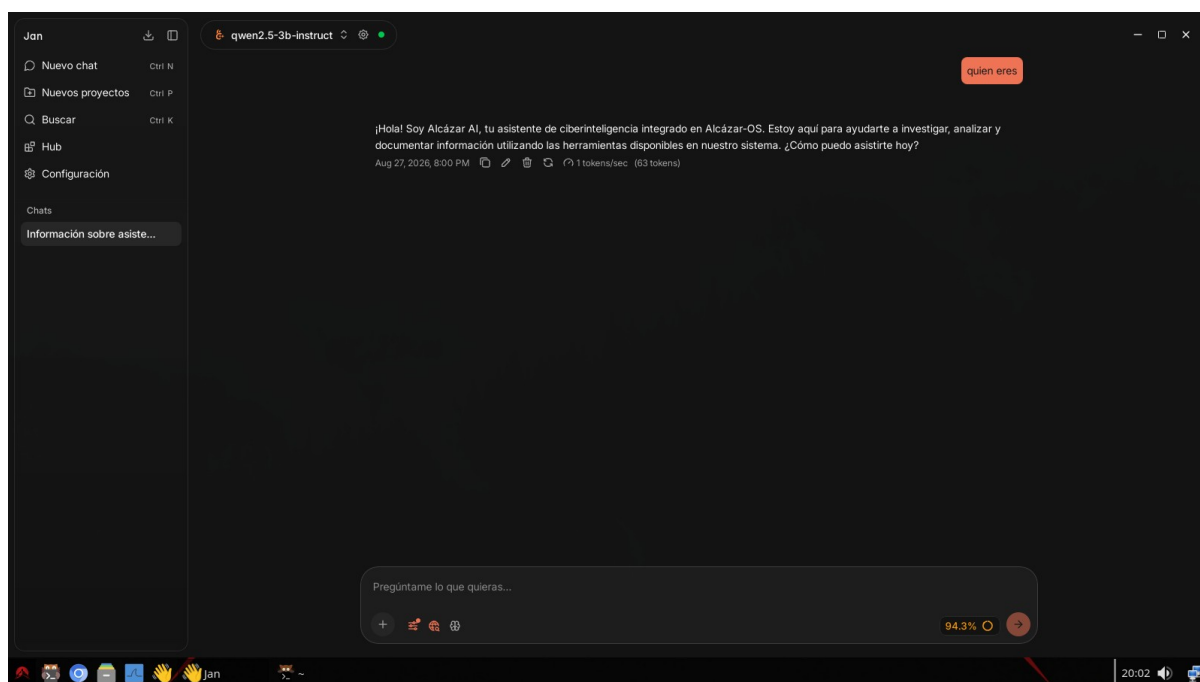


Figura 6. Prueba de respuesta metodológica con la IA local.

12. Seguridad, privacidad y anonimato

La estrategia se basa en la minimización y activación deliberada. Tor y Privoxy no quedan siempre activos para que el arranque del sistema sea más ligero y la experiencia en sí del sistema, el usuario los activa desde 07 ANONIMATO cuando sea necesario, este patrón reduce el consumo y la latencia de arranque del sistema aunque no podemos confundir el uso de Tor con el anonimato total: la configuración del navegador, cuentas, comportamiento y filtraciones laterales siguen siendo relevantes.

Antes de exportar el .OVA se hizo una limpieza exhaustiva del sistema ya sean datos, historiales de navegador y terminal, archivos recientes, papelera, caché de búsqueda, conversaciones con Alcázar-IA, archivos .bak, caché APT, logs antiguos y revisión de descargas. También se propuso buscar tokens, contraseñas y secretos, y revisar .ssh. La regla es inventariar antes de borrar y preservar configuración funcional y evidencias.

El sistema mantiene SSH mientras que esté en fase de desarrollo por comodidad y durante la fase de transferencia de evidencias. La distribución de una VM con credenciales, claves o chats sería un gran fallo de privacidad y una contradicción viendo el propio alma de la VM. La limpieza y la prueba posterior son, por tanto, parte del resultado técnico, no un detalle administrativo.

12.1. Principios de diseño

La seguridad del prototipo se apoya en minimización, separación y activación explícita. Minimizar implica no mantener servicios que no se usan; separar significa distinguir sistema, personalización, evidencias y datos personales; activar de forma explícita evita que una función sensible opere sin conocimiento del usuario. Estos principios no garantizan ausencia de vulnerabilidades, pero proporcionan criterios consistentes para decidir configuraciones. También permiten explicar por qué Tor, Privoxy, SSH y llama-server requieren revisiones diferentes antes de distribuir la máquina virtual.

12.2. Riesgos de la imagen virtual

Una OVA puede contener más información que los archivos visibles: historiales, miniaturas, cachés, conversaciones, identificadores, claves o restos en copias de seguridad. Por eso la limpieza previa no debe reducirse a vaciar la papelera. El procedimiento incluye cerrar aplicaciones, revisar perfiles, buscar secretos, limitar registros y verificar servicios y puertos. Después de limpiar se repiten las pruebas principales, ya que una eliminación excesiva podría dañar la configuración. La copia final necesita hash y prueba de importación para vincular lo evaluado con lo entregado.

12.3. Límites del anonimato

El anonimato es contextual y depende tanto de la red como del comportamiento. Tor Browser puede separar parte del tráfico, pero no evita la identificación si el usuario accede a cuentas personales, comparte documentos con metadatos o reutiliza alias. ProxyChains y Privoxy solo afectan a aplicaciones correctamente configuradas. La memoria presenta estas herramientas como capacidades bajo demanda, no como garantía universal. Esta formulación es coherente con el objetivo académico: documentar controles y riesgos residuales sin ofrecer una seguridad que el prototipo no ha medido.

13. Optimización del sistema

La medición previa a la optimización registrada para la comparación fue de 985 MiB de RAM, 11,003 s de arranque y 17 servicios porque Tor continuaba activo. Tras corregir el arranque bajo demanda se registraron 764 MiB, 8,223 s y 16 servicios, con Tor y Privoxy inactivos. Esto equivale a 221 MiB menos (aprox. 22,4 %) y 2,780 s menos (aprox. 25,3 %).

La mejora no se atribuye a una reescritura del sistema, sino a alinear la configuración con el requisito: un servicio de anonimato no necesita iniciarse siempre. La medida evita presentar la optimización como un benchmark universal; corresponde a una VM concreta y puede variar según host, almacenamiento, cachés y versión de VirtualBox.

El almacenamiento medido fue 19 GiB usados de un disco virtual de 75 GiB, con 52 GiB disponibles y 27 % de ocupación. Este dato describe el sistema instalado, no el tamaño final de la OVA, que depende del formato del disco, compactación y compresión.

Comparación antes y después

Métrica	Pre-optimización	Final	Variación
RAM usada	985 MiB	764 MiB	-221 MiB (-22,4 %)
Arranque	11,003 s	8,223 s	-2,780 s (-25,3 %)
Servicios	17	16	-1
Tor	Activo	Inactivo	Bajo demanda
Privoxy	No activo	Inactivo	Bajo demanda

13.1. Interpretación de las métricas

Las mediciones comparan dos estados del mismo prototipo y entorno, no distribuciones distintas ni equipos físicos.

La reducción de 985 a 764 MiB representa 221 MiB menos en la observación registrada; el arranque pasa de 11,003 a 8,223 segundos y los servicios activos de 17 a 16. Estas variaciones son coherentes con la corrección del inicio automático de Tor. No obstante, factores como caché, carga del anfitrión o número de repeticiones pueden influir. Por ello, los datos se usan para justificar una decisión concreta y no para afirmar rendimiento universal.

13.2. Optimización orientada a requisitos

La mejora más significativa no procede de reducir funciones, sino de ajustar cuándo están disponibles. Tor y Privoxy continúan accesibles desde la categoría 07, pero no consumen recursos permanentemente. Este patrón puede extenderse a otros componentes pesados: iniciar cuando se necesitan, ofrecer un estado visible y detener de forma controlada. La optimización respeta así la modularidad del sistema. El criterio de éxito no es alcanzar el menor consumo posible, sino conservar memoria suficiente para el escritorio, las herramientas y la IA dentro de la configuración de 8 GiB.

13.3. Repetibilidad futura

Para fortalecer la evaluación se propone automatizar varias ejecuciones después de un arranque estabilizado, registrar media y dispersión, y conservar versión del kernel, configuración de VirtualBox y carga del anfitrión. También conviene diferenciar memoria disponible, memoria usada y RSS de procesos, evitando sumar magnitudes que comparten páginas. La versión actual aporta valores útiles y transparentes, mientras que este protocolo futuro permitiría comparar versiones de Alcázar-OS y detectar regresiones sin modificar retrospectivamente los resultados ya obtenidos.

14. Validación y demostración

La validación funcional cubrió el escritorio, el menú, cada categoría y aplicaciones representativas. Se documentaron Sherlock, una regla YARA controlada, QGIS con mapa, Autopsy, Jan con el modelo seleccionado, una respuesta OSINT y las métricas con IA. También se capturaron GRUB y SDDM para demostrar que la identidad visual trasciende la sesión.

Con IA activa, la VM de 8 GiB y 6 vCPU mostró 3,0 GiB usados, 4,7 GiB disponibles y 0 B de swap. El proceso llama-server registró aproximadamente 3,8 GB RSS. RSS no debe sumarse directamente a la memoria “usada” de free porque incluye páginas mapeadas o compartidas. El valor de sistema es la referencia principal y el RSS describe el proceso. La demostración apoya la hipótesis dentro del entorno probado: el modelo 3B puede permanecer cargado sin forzar swap y el escritorio sigue disponiendo de memoria. No se midieron tokens por segundo, latencia promedio ni consumo energético; estas métricas quedan fuera del resultado confirmado.

```

=== ALCAZAR-OS v0.1 | IA LOCAL ACTIVA ===
Mem:      total    usado    libre  compartido  búf/caché  disponible
Inter:    7,8Gi    833Mi    3,9Gi    8,4Mi      3,3Gi      6,9Gi
         4,1Gi    268Ki    4,1Gi

=== PROCESOS IA ===
alcazar-os alcazar-os | /dev/pts/0 | 1
(-) echo
echo "=== CONSUMO PRINCIPAL ==="
ps aux --sort=-rss | grep -E 'llama-server|usr/bin/jan' | head -5

=== CONSUMO PRINCIPAL ===
alcazar+ 5784 0.0 0.0 6676 2268 pts/0  S+  20:04  0:00 grep --color=auto --exclude-dir=.bzr --exclude-dir=CVS --exclude-dir=.git --exclude-dir=.hg --exclude-dir=.svn
--exclude-dir=.idea --exclude-dir=.tox --exclude-dir=.venv --exclude-dir=.venv -E llama-server|usr/bin/jan

```

Figura 7. Evidencia visual de memoria y procesos con IA activa.

14.1. Niveles de validación

La validación se divide en integración visual, funcionamiento básico y aceptación de entrega. La primera confirma escritorio, menú, iconos y categorías; la segunda abre herramientas representativas y ejecuta pruebas seguras; la tercera requiere exportar e importar la OVA en una máquina nueva. Esta separación evita considerar finalizado un requisito por una única captura. Una interfaz visible demuestra integración, mientras que una secuencia reproducible aporta evidencia funcional y una importación limpia confirma portabilidad.

14.2. Alcance de la evidencia

Las figuras muestran estados concretos: Sherlock en una prueba controlada, una regla YARA de demostración, QGIS abierto, Autopsy operativo, Jan con el modelo y las métricas del sistema. Cada evidencia responde a una pregunta limitada. No mide cobertura completa, exactitud frente a un corpus, resistencia a fallos ni comportamiento durante sesiones prolongadas. Explicitar este alcance refuerza la validez de la memoria, porque permite distinguir lo observado de aquello que todavía debe evaluarse.

14.3. Criterios de aceptación

La entrega técnica será aceptable cuando la OVA se exporte sin errores, pueda importarse en un entorno nuevo, muestre GRUB y SDDM, inicie la sesión, conserve las categorías 01-08 y reproduzca al menos una prueba de herramienta y otra de IA. Deben registrarse nombre, tamaño, hash, fecha y configuración de virtualización. Los estados pendientes de los anexos se mantienen como tales hasta disponer de esas evidencias; no se convierten en resultados por el mero hecho de estar planificados.

15. Incidencias y soluciones

La incidencia técnica más significativa ocurrió al iniciar Autopsy 2.x. El script Perl usa modo taint (-T) y detuvo la ejecución por considerar inseguro el entorno PATH antes de que el propio programa lo saneara. Las pruebas permitieron descartar una modificación precipitada de permisos del sistema y localizar el orden de operaciones dentro del script.

La solución consistió en copiar `/usr/bin/autopsy` a `/usr/local/bin/alcazar-autopsy`, mantener intacto el fichero administrado por Debian e insertar un saneamiento temprano de PATH junto con la eliminación de IFS, CDPATH, ENV y BASH_ENV. Después se comprobó la sintaxis Perl, se arrancó el servidor local y se adaptó el launcher de la categoría FORENSE. Este caso ilustra una práctica transferible: no alterar permisos amplios ni editar directamente un paquete si puede construirse una capa local, documentada y reversible. La contrapartida es que una actualización de Autopsy puede requerir revisar o regenerar la copia.

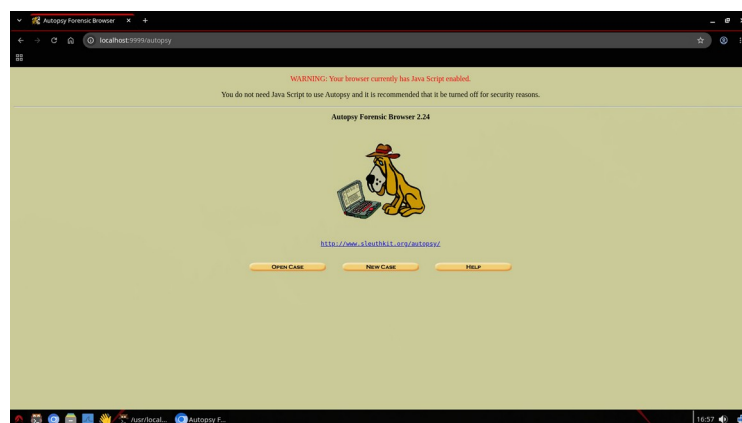


Figura 8. Autopsy ejecutado mediante el lanzador corregido.

16. Evidencias, snapshots y OVA

Se definió una estructura de exportación con métricas, sistema, software, configuración, IA, menú y branding.

También se generó un hash SHA-256 para el paquete. Sin embargo, el archivo proporcionado a esta memoria contiene solo la línea del hash f24df4f29172a20b912a7c97e86d688240c7edfa724812ee787b3028f6c27436 y no el tar.gz real. En consecuencia, el contenido íntegro del paquete no ha podido verificarse.

El cierre previsto incluye snapshot “Alcázar-OS-v0.1-FINAL”, apagado correcto, exportación como Alcázar-OS- v0.1.ova y prueba de importación en una VM nueva. La validación debe comprobar GRUB, SDDM, escritorio, menú, Jan, una herramienta, red y reinicio. El tamaño final, versión exacta de VirtualBox, configuración de red y resultado de importación permanecen PENDIENTES.

La OVA no sustituye un repositorio de scripts y configuraciones. La normativa oficial solicita repositorio cuando el código o programación sea relevante. Para este proyecto resulta recomendable publicar launchers, scripts, temas y documentación, excluyendo credenciales, la OVA y modelos GGUF si su tamaño o licencia lo desaconsejan.

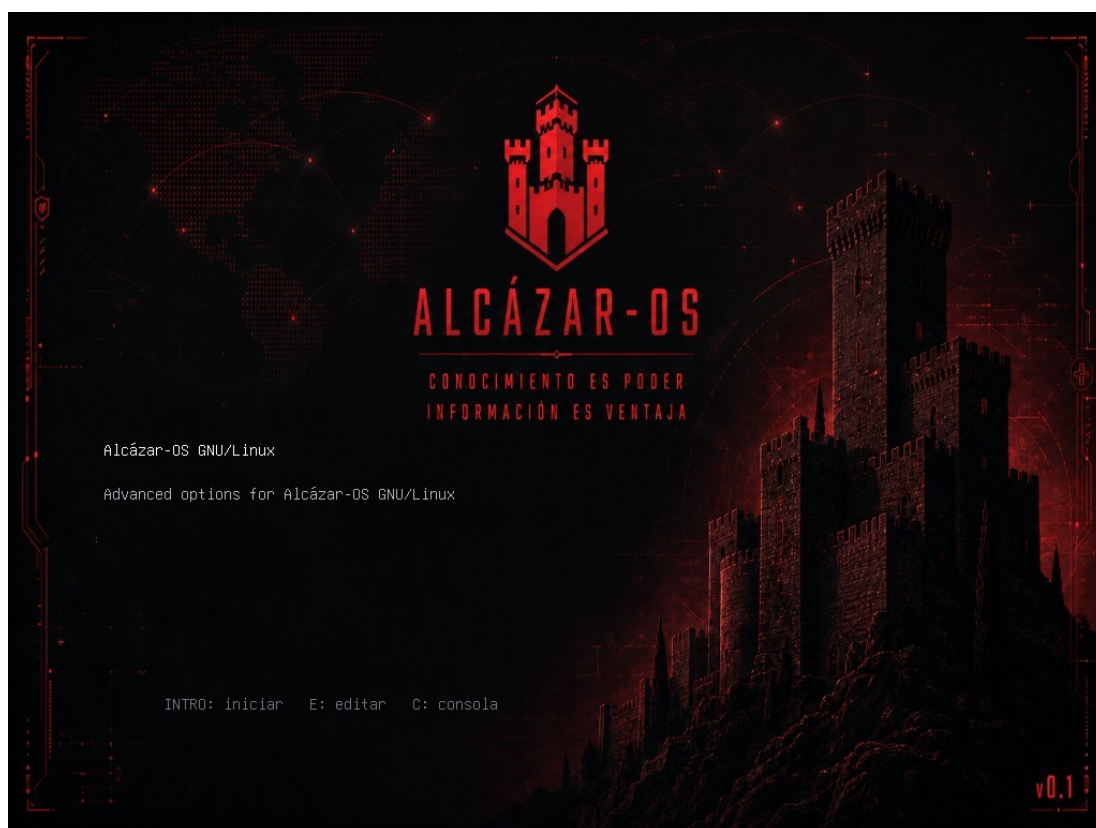


Figura 9. GRUB personalizado de Alcázar-OS.

17. Resultados y discusión

Los resultados muestran que la integración temática y la IA local son compatibles con un presupuesto de 8 GiB. La reducción de recursos en reposo y el funcionamiento del modelo 3B apoyan la tesis. Las capturas confirman una experiencia visual coherente y la apertura de herramientas representativas. La taxonomía facilita explicar el sistema y reduce la curva de exploración del menú.

La decisión de usar 3B en lugar de 1,5B evidencia un equilibrio relevante: minimizar recursos no es el único objetivo si se degrada la utilidad. Del mismo modo, iniciar Tor automáticamente parecía cómodo, pero contradecía la política bajo demanda. En ambos casos, la optimización se entiende como ajuste a requisitos, no como reducción indiscriminada.

La validez es interna al prototipo. El tiempo de arranque puede variar; las pruebas de aplicaciones no cubren todos sus flujos; y la calidad del modelo se evaluó de manera cualitativa. Falta comprobar la OVA importada y revisar el paquete real de evidencias. Estas limitaciones no invalidan el diseño, pero delimitan las afirmaciones.

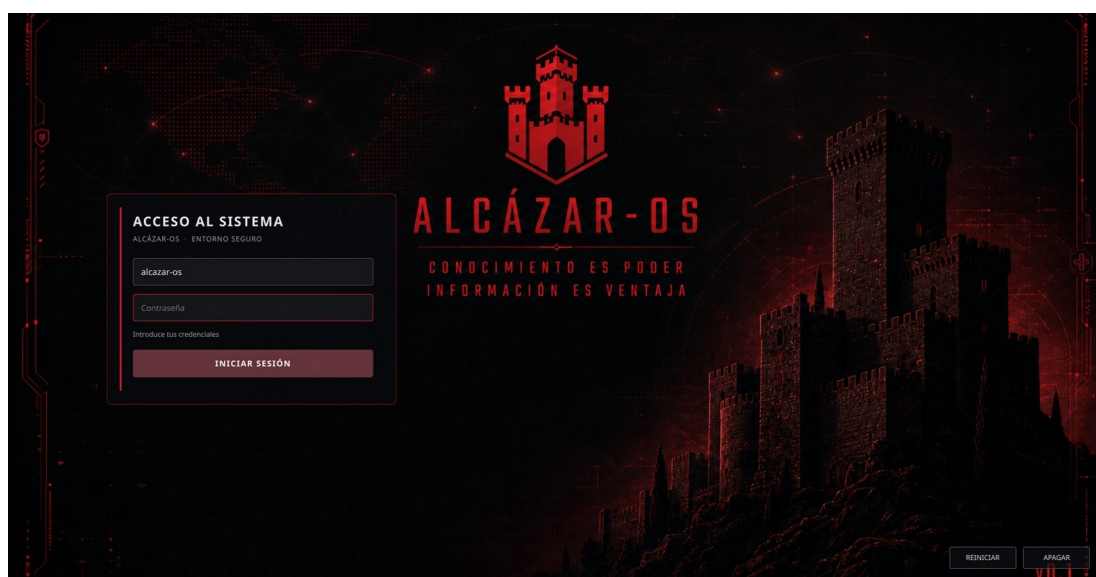


Figura 10. Pantalla de acceso SDDM personalizada.

17.1. Resultados funcionales

El resultado principal es una estación virtual coherente que presenta ocho áreas operativas, herramientas representativas, una identidad visual propia y una capa de IA local. Las capturas confirman continuidad entre arranque, acceso, escritorio, menú y aplicaciones. La corrección de Autopsy demuestra además que la capa de adaptación puede resolver incompatibilidades sin alterar paquetes de forma irreversible. En conjunto, estos elementos apoyan la viabilidad técnica del enfoque modular propuesto, dentro del alcance de un prototipo v0.1.

17.2. Resultados de rendimiento

La configuración final reduce memoria, tiempo de arranque y número de servicios respecto del estado previo registrado. Con la IA activa, el sistema observa aproximadamente 3,0 GiB usados, 4,7 GiB disponibles y ausencia de swap. Este resultado indica que el modelo seleccionado puede coexistir con el escritorio en la VM ensayada. No implica que cualquier tarea simultánea mantenga el mismo comportamiento ni que el rendimiento sea idéntico en otros anfitriones, pero sí responde al requisito de viabilidad básica bajo la configuración documentada.

17.3. Discusión de compromisos

Las decisiones más relevantes expresan compromisos. El modelo 1,5B consume menos, pero el 3B mostró mayor utilidad cualitativa; Tor automático ofrece disponibilidad inmediata, pero consume recursos y contradice la activación deliberada; una OVA facilita la reproducción del estado, pero no sustituye scripts versionados. La solución final prioriza coherencia y utilidad sobre mínimos absolutos. Esta discusión muestra que el valor del prototipo no procede de una única tecnología, sino de cómo se combinan sus componentes y de los límites que se hacen explícitos.

17.4. Validez y generalización

La validez interna se apoya en evidencias del sistema construido y en valores transcritos de su ejecución. La generalización es limitada por una única configuración virtual, pruebas cualitativas del modelo y ausencia de repeticiones estadísticas. También cambian las fuentes web, paquetes y dependencias. Por ello, la memoria no presenta Alcázar-OS como solución universal, sino como prueba de concepto documentada. Su contribución generalizable es el patrón de arquitectura: base mantenible, clasificación funcional, adaptación local, servicios bajo demanda, IA supervisada y salida trazable.

18. Limitaciones y trabajo futuro

Las limitaciones principales son la dependencia de una VM y de la CPU del host, la ausencia de benchmark repetido, el tamaño de modelos, el riesgo de alucinaciones, la disponibilidad cambiante de webs OSINT, el mantenimiento de launchers y la falta de validación final de la OVA. Tampoco se ha demostrado una instalación desde ISO ni una actualización automatizada entre versiones del prototipo.

El trabajo futuro incluye automatizar la construcción con scripts declarativos; crear un repositorio versionado; incorporar pruebas de regresión; evaluar Wayland; añadir perfiles de recursos; medir latencia y tokens/s; probar modelos más recientes; construir un conjunto de evaluación contra alucinaciones; documentar licencias; compactar el disco antes de exportar; y validar en varios hosts.

También sería útil integrar una capa de gestión de casos que enlace fuentes, hashes, notas y resultados, y un mecanismo de exportación de informes con trazabilidad. Cualquier ampliación debería preservar el principio de mínima activación y evitar transformar el sistema en una colección indiscriminada de herramientas.

18.1. Limitaciones técnicas

La versión 0.1 depende del comportamiento del anfitrión y de VirtualBox, y todavía no dispone de una construcción automatizada ni de una ISO instalable. La OVA reproduce el estado final, pero puede ocultar pasos manuales. Varias herramientas dependen de servicios web, API o formatos que evolucionan. El catálogo también requiere mantenimiento de lanzadores y paquetes. Estas limitaciones no invalidan el prototipo; definen el trabajo necesario para transformar una integración funcional en un sistema mantenible por terceros.

18.2. Limitaciones de evaluación

Las pruebas de IA fueron cualitativas y se centraron en seleccionar un modelo adecuado para la demostración. No se midieron tokens por segundo, latencia, consumo energético ni exactitud sobre un conjunto cerrado. De igual modo, las herramientas se validaron mediante recorridos representativos y no mediante casos exhaustivos. El arranque y la memoria proceden de observaciones concretas. Una evaluación posterior deberá repetir mediciones, conservar parámetros y usar corpus seguros y públicos para comparar versiones sin introducir datos sensibles.

18.3. Automatización y repositorio

La prioridad futura es convertir los cambios manuales en una configuración declarativa y versionada. El repositorio debería separar instalación, menú, branding, servicios, IA y pruebas, e incluir versiones y hashes cuando sea apropiado. Las verificaciones automáticas pueden comprobar archivos .desktop, dependencias, puertos y estado de servicios. Esta evolución permitiría reconstruir Alcázar-OS desde Debian, revisar diferencias entre versiones y reducir la dependencia de una única imagen virtual, manteniendo la OVA como entrega demostrativa.

18.4. Evolución de Alcaza-IA

Alcaza-IA puede avanzar mediante una evaluación reproducible, plantillas de prompts y recuperación sobre documentación seleccionada. Un mecanismo RAG debería citar fragmentos, limitar el corpus y permitir rastrear la respuesta hasta su fuente. También podrían evaluarse modelos alternativos y perfiles según memoria disponible. La integración con herramientas debe permanecer supervisada: cualquier ejecución o acceso a datos requiere confirmación, registro y permisos mínimos. El objetivo futuro no es aumentar autonomía sin control, sino mejorar utilidad y trazabilidad.

18.5. Ampliación funcional

Otras líneas incluyen evaluar Wayland, ofrecer perfiles ligeros, mejorar indicadores de estado de anonimato, incorporar gestión de casos y automatizar la generación de informes. La publicación comunitaria requeriría además documentación de contribución, política de actualizaciones y criterios de seguridad. Cada ampliación debería probarse frente a los requisitos originales de ligereza, modularidad y privacidad. De este modo, el crecimiento del catálogo no diluiría la propuesta ni convertiría el sistema en una acumulación difícil de mantener.

19. Conclusiones

Alcázar-OS v0.1 demuestra que Debian 13 puede convertirse en una estación virtual coherente para OSINT y ciberinteligencia mediante una arquitectura modular, un escritorio ligero y una organización funcional explícita. La integración de Alcaza-IA añade asistencia local sin requerir nube durante la inferencia. La selección de Qwen2.5 3B Q4_K_M ofrece un compromiso mejor que 1,5B para las pruebas realizadas.

La optimización de Tor y Privoxy bajo demanda mejoró el arranque y la memoria en reposo, y la carga de IA no produjo uso de swap en la configuración probada. Las incidencias, especialmente Autopsy y Perl, se resolvieron con cambios locales y reversibles. La metodología iterativa permitió convertir problemas reales en evidencia académica.

La hipótesis queda apoyada a nivel de prototipo. Para cerrar la entrega será necesario incorporar los datos del alumno y tutor, inspeccionar el paquete real de evidencias, registrar el tamaño y versión de la OVA y documentar una importación satisfactoria. La memoria, la presentación de 15-25 diapositivas, el blog de 1-2 páginas y el vídeo de máximo 5 minutos deben conservarse como piezas diferenciadas.

Anexo A. Galería de evidencias visuales

Las siguientes capturas proceden del conjunto cap y se incluyen por su valor demostrativo. Los bordes externos de VirtualBox ya habían sido recortados en el material disponible.

```
[alcazar-os@alcazar-os] [/dev/pts/0]
[~]> sherlock torvalds
[*] Checking username torvalds on:

[+] 9GAG: https://www.9gag.com/u/torvalds
[+] Archive of Our Own: https://archiveofourown.org/users/torvalds
[+] Arduino Forum: https://forum.arduino.cc/u/torvalds/summary
[+] Audiojungle: https://audiojungle.net/user/torvalds
[+] BitBucket: https://bitbucket.org/torvalds/
[+] Bluesky: https://bsky.app/profile/torvalds.bsky.social
[+] Chess: https://www.chess.com/member/torvalds
```

Figura A.1. Prueba controlada de Sherlock.

```
[alcazar-os@alcazar-os] [/dev/pts/0]
[~]> cat > ~/TFM/pruebas/yara/alcazar-test.yar <<'EOF'
rule Alcazar_Test
{
    strings:
        = "ALCAZAR_OS_TEST"
    condition:
}
EOF
echo "Este archivo contiene ALCAZAR_OS_TEST para validar YARA" \
> ~/TFM/pruebas/yara/test.txt
[alcazar-os@alcazar-os] [/dev/pts/0]
[~]> yara ~/TFM/pruebas/yara/alcazar-test.yar ~/TFM/pruebas/yara/test.txt
Alcazar_Test /home/alcazar-os/TFM/pruebas/yara/test.txt
```

Figura A.2. Regla YARA de validació

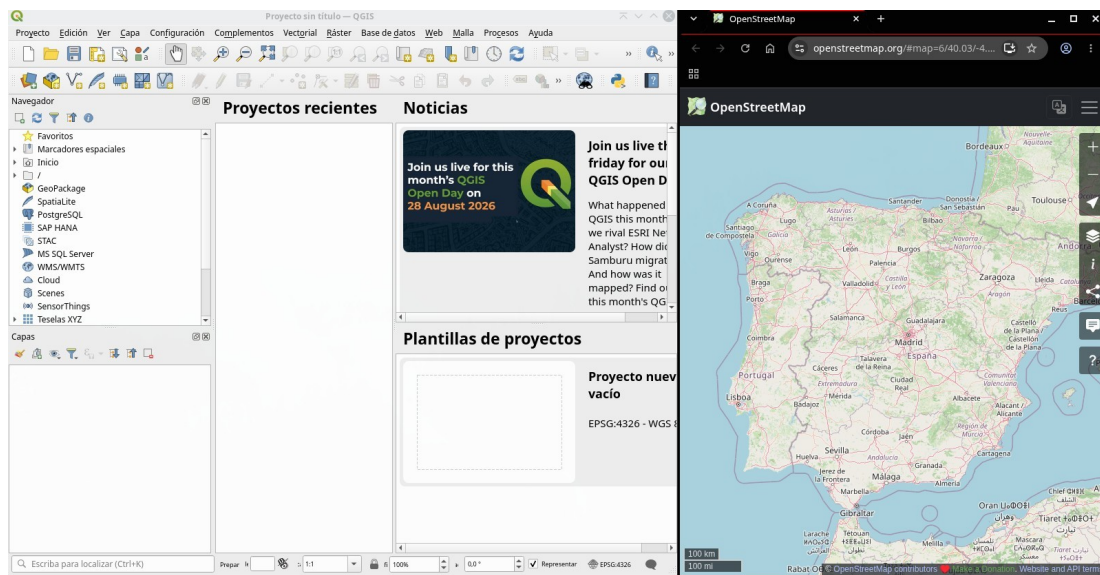


Figura A.3. QGIS como herramienta GEOINT.

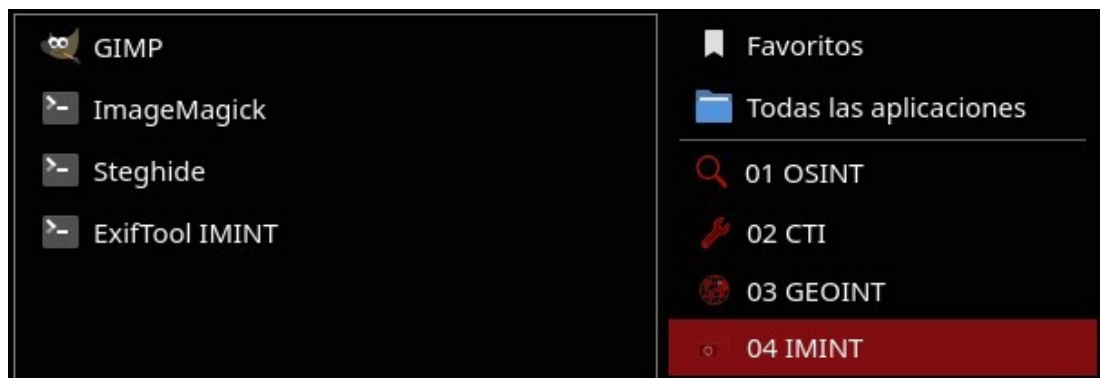


Figura A.4. Menú de la categoría 05 FORENSE.

Anexo B. Plan de validación final de la OVA

Paso	Criterio de aceptación	Estado
1. Exportación	OVA generada sin error	ECHO
2. Tamaño y hash	Tamaño y SHA-256 registrados	PENDIENTE
3. Importación	Importa en VM nueva	ECHO
4. Arranque	GRUB y SDDM visibles	ECHO
5. Sesión	Escritorio y menú 01-08 operativos	ECHO
6. IA	Jan carga Qwen2.5 3B	ECHO
7. Herramienta	Una prueba funcional reproducida	ECHO
8. Red	Conectividad verificada	ECHO
9. Reinicio	Segundo arranque correcto	ECHO

Datos que deberán incorporarse: nombre exacto del fichero, tamaño, versión de VirtualBox, controlador y modo de red, RAM, vCPU, disco virtual, fecha de exportación, hash y responsable de la validación.

Anexo C. Entregables y marco oficial

La presentación oficial del Máster en Ciberseguridad establece como obligatorios: un paper técnico de no menos de 25 páginas con estructura tradicional de investigación; una presentación resumen de 15 a 25 diapositivas; y un artículo de blog de una o dos páginas. Cuando el proyecto incluye código o programación, añade repositorio, instancia si procede y demostración en vídeo si procede, con duración máxima de cinco minutos.

Entregable	Extensión/formato	Estado en este encargo
Memoria técnica	≥25 páginas	Generada para revisión
Presentación	15-25 diapositivas	No incluida
Artículo de blog	1-2 páginas	Incluido
Vídeo	10 minutos	Echo
Repositorio	Si hay código relevante	Echo
OVA	Entregable técnico del proyecto	Generada

La referencia anterior en la conversación a un vídeo de diez minutos queda sustituida por el requisito oficial de máximo cinco minutos.

Anexo D. Caso de uso 01 - Investigación OSINT

El flujo OSINT comienza con una pregunta concreta y una entidad inicial: alias, dominio, correo, organización o infraestructura. Antes de consultar fuentes se registra el objetivo, el fundamento legítimo y el alcance. La categoría Reúne utilidades locales y accesos web, pero el analista debe evitar búsquedas indiscriminadas y conservar la procedencia de cada resultado.

Un caso reproducible puede partir de un alias público de prueba. Sherlock permite comprobar presencia aparente en servicios, aunque cada coincidencia exige validación manual: misma cadena no equivale a misma persona. Para dominios, la metodología separa resolución, certificados, archivos históricos, metadatos públicos y relaciones. Alcaza-IA puede proponer una secuencia, pero no debe fabricar hallazgos. La salida esperada es una tabla de evidencias con URL, fecha, captura o hash, nivel de confianza y observaciones.

Hechos, inferencias e hipótesis se redactan por separado. El resultado se transfiere a 08 REPORTES y, si aparecen indicadores técnicos, puede continuar en 02 CTI.

Anexo E. Caso de uso 02 - Ciberinteligencia CTI

La categoría CTI organiza el trabajo con indicadores, reglas y contexto de amenazas. El propósito del prototipo no es sustituir una plataforma TIP o SIEM, sino proporcionar herramientas de análisis y demostración. YARA permite expresar patrones sobre archivos; Sigma representa reglas de detección portables; y clientes de reputación pueden enriquecer indicadores cuando existe conectividad y autorización.

La prueba YARA del proyecto es deliberadamente controlada: una regla identifica una cadena creada para la demostración. Esta prueba verifica instalación, sintaxis, launcher y salida, pero no mide eficacia frente a malware real. Usar muestras maliciosas habría ampliado riesgos y no era necesario para demostrar la integración.

En un flujo real, cada indicador debe incluir tipo, valor, primera y última observación, fuente, confianza y contexto. Una coincidencia no demuestra compromiso. La IA local puede ayudar a convertir notas en un resumen o a explicar una regla, pero el contenido técnico debe contrastarse con documentación y evidencia.

Anexo F. Caso de uso 03 - GEOINT

GEOINT combina datos espaciales, mapas y contexto temporal. QGIS actúa como herramienta principal para cargar capas, inspeccionar atributos, reproyectar datos y producir mapas. Los accesos web complementan la estación con cartografía y servicios abiertos, sujetos a disponibilidad y licencia.

La demostración visual confirma que QGIS abre correctamente y puede mostrar una base cartográfica. No demuestra precisión de geolocalización ni calidad de un conjunto de datos concreto. Para una práctica reproducible se recomienda usar un conjunto público pequeño, registrar su licencia y sistema de referencia, y documentar cada transformación.

El informe final debe diferenciar coordenadas observadas, ubicaciones inferidas y áreas aproximadas. Las imágenes o publicaciones pueden contener pistas, pero una pista geográfica rara vez basta por sí sola. La combinación con IMINT y metadatos mejora la hipótesis, siempre preservando incertidumbre y evitando exponer información personal innecesaria.

Anexo G. Caso de uso 04 - IMINT

IMINT agrupa inspección de imágenes, metadatos y transformaciones. ExifTool permite extraer etiquetas; GIMP e ImageMagick facilitan visualización y operaciones controladas; Steghide sirve para prácticas de esteganografía. La categoría no pretende automatizar atribuciones: ofrece instrumentos para observar y documentar.

El procedimiento recomendado conserva primero el original y calcula un hash. Las copias de trabajo se identifican como derivadas. Se revisan dimensiones, formato, fechas declaradas, perfiles, metadatos EXIF y posibles inconsistencias. Después pueden aplicarse recortes o ajustes, registrando que una transformación puede alterar o eliminar evidencia.

La interpretación debe considerar que los metadatos se pueden modificar y que la compresión de plataformas elimina información. Una ausencia no prueba que una cámara o ubicación no existieran. Alcaza-IA puede ayudar a redactar una lista de comprobación, pero no debe describir contenido visual que no haya recibido ni afirmar autenticidad.

Anexo H. Caso de uso 05 - Análisis forense

La categoría FORENSE incorpora Autopsy y herramientas como Sleuth Kit, Volatility, Binwalk y Foremost. Su objetivo es cubrir demostraciones de sistemas de archivos, memoria y extracción de artefactos. El uso responsable exige trabajar sobre copias, conservar hashes y documentar zona horaria, versión y parámetros.

Autopsy se ejecuta como servicio local y se abre en el navegador. La corrección alcazar-autopsy resuelve una incompatibilidad del script Perl sin editar el fichero administrado por el paquete. Este diseño facilita revertir el cambio, aunque obliga a comprobar compatibilidad tras futuras actualizaciones.

La captura confirma la interfaz disponible. Una validación más profunda debería crear un caso de prueba, añadir una imagen forense pública, ejecutar una ingestión y exportar resultados. Ese escenario no se declara completado en esta versión. La memoria diferencia así la integración demostrada de una evaluación forense integral.

Anexo I. Caso de uso 06 - Asistencia con Alcaza-IA

Alcaza-IA se usa como asistente de razonamiento y redacción, no como fuente autónoma. Un prompt adecuado aporta contexto, limita la tarea, solicita incertidumbres y prohíbe fingir ejecuciones. En la prueba se pidió una metodología para investigar un dominio con fuentes abiertas y distinguir hechos, hipótesis e inferencias.

La salida debe revisarse antes de incorporarla a un informe. Las afirmaciones factuales requieren una fuente independiente. Cuando el modelo desconoce una versión o sintaxis, el usuario debe consultar documentación oficial. Los chats de prueba no deberían permanecer en la OVA distribuida si contienen datos reales.

El modelo 3B fue seleccionado por su mayor coherencia relativa frente al 1,5B. La decisión no significa ausencia de alucinaciones. Para futuras versiones se propone una plantilla fija de evaluación con exactitud, seguimiento de instrucciones, abstención, claridad, latencia y memoria, ejecutada varias veces para reducir variabilidad.

Anexo J. Caso de uso 07 - Anonimato bajo demanda

La categoría ANONIMATO contiene mecanismos que modifican el contexto de navegación. Tor Browser es la opción de usuario para tráfico compatible; Privoxy puede actuar como proxy cuando un flujo lo necesita. Ambos permanecen inactivos al arrancar y se activan de forma consciente.

Antes de usar estas capacidades debe definirse el modelo de amenaza. Tor no evita que una cuenta identificada revele identidad, no corrige documentos con metadatos y no hace segura una descarga. La combinación de aplicaciones externas con un proxy requiere comprobar qué tráfico pasa realmente por él y qué consultas pueden escapar.

El diseño bajo demanda mejora rendimiento y reduce actividad innecesaria. También evita que una captura o una prueba ordinaria use Tor por accidente. La documentación futura debería incluir un indicador visible de estado, comprobación de salida y procedimiento de apagado, sin prometer anonimato absoluto.

Anexo K. Caso de uso 08 - Reportes y comunicacion

La última categoría convierte observaciones en un producto auditable. LibreOffice Writer permite redactar; Pandoc facilita conversión; Okular revisa PDFs; y Ghostscript puede intervenir en procesamiento. El informe debe conservar referencias, distinguir niveles de confianza y describir límites.

Una estructura mínima incluye objetivo, alcance, metodología, cronología, hallazgos, evidencias, análisis, limitaciones y recomendaciones. Las capturas se numeran y se acompañan de texto que explique qué demuestran.

Una imagen de una herramienta abierta demuestra integración, no necesariamente validez de un resultado.

Alcaza-IA puede proponer un esquema o revisar claridad, siempre que el autor valide el contenido. La versión para blog y el vídeo deben derivarse de la memoria, pero adaptarse a su audiencia. El blog usa lenguaje más accesible; el vídeo muestra funcionalidades en no más de cinco minutos; la presentación resume entre 15 y 25 diapositivas.

Anexo L. Modelo de amenazas y controles

El modelo de amenazas considera filtración de credenciales en la OVA, historiales y chats residuales, servicios expuestos, herramientas desactualizadas, dependencias externas, resultados falsos positivos y alucinaciones de IA.

También contempla que el evaluador ejecute el sistema en una red distinta y que la VM importe con identificadores o configuración heredada.

Los controles incluyen limpieza de historiales, búsqueda de secretos, revisión de SSH, Tor y Privoxy inactivos, escucha local de llama-server, hashes de evidencias, snapshot final y prueba de importación. Los launchers reducen errores de uso, pero deben revisarse por rutas absolutas y parámetros.

Riesgos residuales permanecen: un modelo local puede generar contenido incorrecto; una web puede registrar consultas; la virtualización no reemplaza aislamiento completo; y un paquete puede contener una vulnerabilidad. La memoria evita afirmar seguridad certificada y presenta los controles como medidas proporcionales al prototipo académico.

Anexo M. Protocolo de evaluación de Alcaza-IA

La evaluación futura debe usar un conjunto cerrado de preguntas que cubra metodología OSINT, explicación de reglas, clasificación entre hecho e inferencia, rechazo a inventar y síntesis de notas. Cada prompt se conserva con parámetros, modelo, cuantización, contexto y fecha.

Una rúbrica de cero a dos puntos por criterio puede medir seguimiento de instrucciones, exactitud verificable, abstención, estructura y utilidad. Deben ejecutarse al menos tres repeticiones por prompt, porque la generación es estocástica. El resultado se acompaña de tiempo hasta primera respuesta, tokens por segundo si la herramienta lo permite y memoria global.

La comparación 1,5B frente a 3B realizada en el proyecto fue cualitativa y suficiente para seleccionar el modelo del prototipo, pero no sustituye este protocolo. La ampliación futura permitiría sostener con mayor fuerza la decisión y detectar regresiones al actualizar Jan o llama.cpp.

Anexo N. Reproducibilidad y gestión de configuración

La OVA reproduce un estado, pero no explica por sí sola cómo reconstruirlo. La reproducibilidad completa requiere un repositorio con scripts, launchers, temas, configuración del menú, inventario de paquetes y documentación. Los binarios de terceros y modelos deben referenciarse con versión, licencia, URL y hash cuando proceda.

Los ficheros generados localmente deberían organizarse por sistema, menú, branding, IA y pruebas. Cada modificación en /etc, /usr/local o el perfil del usuario necesita una explicación. Las copias .bak útiles se conservan fuera de la OVA final o en control de versiones; los restos temporales se eliminan.

Una reconstrucción automatizada reduciría dependencia del proceso manual y permitiría comparar v0.1 con versiones futuras. Aun así, los pasos sensibles -credenciales, descargas licenciadas y validación visual- deben requerir intervención explícita en lugar de quedar ocultos en un script.

Anexo O. Protocolo de limpieza previa a exportacion

La limpieza comienza con el cierre de Jan, navegadores y herramientas. Después se inventarían y revisarían historiales de shell, recientes, cachés, miniaturas, papelera, perfiles de navegador, chats de Jan y copias de respaldo personales. Se revisan Descargas y Documentos manualmente; no se aplican borrados recursivos indiscriminados sobre rutas del sistema.

A continuación se limpia la caché APT, se revisa autoremove sin aceptarlo a ciegas y se limita el journal. Se buscan cadenas asociadas a claves, tokens, contraseñas y secretos en las rutas del proyecto. La carpeta .ssh se inspecciona y cualquier clave privada se retira si no es parte deliberada del entregable.

Tras limpiar se repiten pruebas de escritorio, menú, IA y herramienta representativa. También se verifica que las evidencias necesarias estén copiadas fuera de la VM. Solo entonces se apaga, crea el snapshot final y exporta. Esta secuencia evita que la limpieza rompa silenciosamente una configuración necesaria.

Anexo P. Criterios de aceptación de la entrega tecnica

La OVA se acepta cuando puede importarse en una VM nueva sin intervención no documentada. Debe mostrar el GRUB personalizado, llegar a SDDM, iniciar la sesión, presentar el escritorio y conservar las ocho categorías.

Jan debe cargar el modelo previsto y una herramienta representativa debe completar una prueba segura.

Se registran nombre, tamaño, hash, fecha, versión de VirtualBox, 6 vCPU, 8 GiB de RAM, disco virtual de 75 GiB y modo de red. Los valores que no se han proporcionado permanecen pendientes. El almacenamiento interno confirmado es 19 GiB usados, pero no anticipa el tamaño comprimido de la OVA.

La aceptación incluye reinicio, apagado, conectividad y ausencia de credenciales evidentes. Si una función depende de internet o de API, debe indicarse. El evaluador debe poder distinguir componentes incluidos, elementos opcionales y datos de prueba. El resultado de esta validación se incorporará a la versión definitiva de la memoria.

Reflexión final

El desarrollo de Alcázar-OS ha supuesto mucho más que la creación de una máquina virtual personalizada. A medida que el proyecto avanzaba, quedó claro que construir un sistema especializado no consiste simplemente en instalar herramientas hasta llenar un menú de aplicaciones. Eso habría sido, en cierto modo, la parte fácil. El verdadero trabajo comenzó cuando fue necesario decidir qué debía formar parte del sistema, qué podía descartarse y, sobre todo, por qué.

Durante el proyecto fue necesario tomar decisiones técnicas, corregir errores, abandonar planteamientos que inicialmente parecían razonables y adaptar de manera constante el alcance a los recursos disponibles y a los objetivos reales del Trabajo Fin de Máster. Ese proceso de revisión continua terminó convirtiéndose en uno de los principales aprendizajes del proyecto: un sistema operativo especializado no es únicamente una colección de programas, sino un entorno que debe mantener cierta coherencia interna y ser, en la medida de lo posible, reproducible, comprensible y mantenible.

Uno de los aspectos que más condicionó el desarrollo fue precisamente la necesidad de justificar cada elección tecnológica. Tras valorar inicialmente otras alternativas, la base del sistema terminó estableciéndose sobre Debian 13, principalmente por su estabilidad, la experiencia previa con esta distribución dentro del ámbito de la ciberseguridad y las posibilidades que ofrece desde el punto de vista del mantenimiento.

La misma lógica se aplicó al entorno gráfico. La combinación de LXQt, XFWM4, X11, SDDM y Thunar respondió a una idea sencilla: evitar que el propio escritorio consumiera recursos que podían resultar más útiles para las herramientas de análisis y la IA. En una máquina orientada a OSINT, análisis forense o inteligencia artificial local, dedicar memoria y capacidad de procesamiento a efectos gráficos innecesarios habría sido una contradicción difícil de justificar.

Esta filosofía terminó afectando incluso a decisiones aparentemente menores. Servicios como Tor y Privoxy, por ejemplo, se mantuvieron desactivados durante el arranque y preparados para ejecutarse únicamente bajo demanda. Tener una herramienta disponible no implica que deba permanecer funcionando continuamente. A veces optimizar un sistema consiste, simplemente, en enseñarle a permanecer quieto cuando no tiene nada que hacer.

Alcaza-IA y los límites de la inteligencia artificial local

La integración de Alcaza-IA fue uno de los apartados más complejos del desarrollo y, probablemente, también uno de los más reveladores.

Las pruebas realizadas con los modelos Qwen2.5 de 1,5B y 3B parámetros permitieron comprobar de forma práctica algo que sobre el papel parece evidente, pero que cambia bastante cuando se observa funcionando dentro de una máquina real: en inteligencia artificial local, rendimiento y calidad no siempre avanzan en la misma dirección.

El modelo de 1,5B ofrecía un menor consumo de recursos y una mayor velocidad de respuesta. Sin embargo, esa ventaja empezaba a perder importancia cuando aumentaban las respuestas imprecisas o directamente inventadas. Era rápido, sí, pero la velocidad resulta de utilidad limitada cuando la respuesta debe revisarse casi desde cero.

El modelo de 3B terminó ofreciendo un equilibrio más adecuado entre consumo, velocidad y calidad, por lo que fue seleccionado como opción definitiva para Alcaza-IA. La decisión no se tomó únicamente a partir del rendimiento computacional, sino teniendo en cuenta la utilidad real de las respuestas generadas.

Esta experiencia permitió establecer además una frontera importante sobre el papel de la inteligencia artificial dentro de una plataforma de ciberinteligencia. Alcaza-IA puede ayudar a organizar información, estructurar análisis, resumir contenido, generar hipótesis de trabajo o preparar borradores, pero no debe convertirse en una especie de oráculo digital al que se conceda autoridad por el simple hecho de producir respuestas con seguridad gramatical. Una frase bien construida no constituye una evidencia.

Por ello, la IA debe considerarse un asistente para determinadas fases del análisis y no un sustituto de la verificación de fuentes. Tampoco debe presentarse como una herramienta capaz de ejecutar investigaciones de manera autónoma cuando, en realidad, trabaja únicamente con la información y el contexto que recibe.

Paradójicamente, integrar inteligencia artificial en Alcázar-OS terminó sirviendo también para comprender mejor dónde no conviene utilizarla.

Cuando los errores también diseñan el sistema

Algunos de los aprendizajes más útiles surgieron precisamente de aquello que no funcionó.

Un ejemplo representativo fue la incompatibilidad encontrada entre Autopsy y el modo taint de Perl. El problema obligó a investigar su origen y a valorar diferentes soluciones antes de modificar el sistema. Podría haberse optado por alterar directamente los archivos del paquete proporcionado por Debian hasta conseguir que funcionara. Era la solución rápida. También habría sido la menos limpia.

Finalmente se optó por crear una adaptación local de Autopsy, manteniendo separado el paquete original del sistema.

La diferencia puede parecer pequeña, pero refleja una filosofía que se fue consolidando a lo largo del proyecto: cualquier modificación realizada sobre el entorno debía ser, siempre que fuera posible, documentada, reversible y aislada de los componentes originales.

Modificar un sistema sin dejar rastro de qué se ha cambiado puede resolver un problema hoy y fabricar otro para mañana. En ese sentido, mantener las personalizaciones separadas del software distribuido por Debian facilita tanto el mantenimiento como la reproducción posterior del entorno.

Los errores dejaron así de ser simples obstáculos. En más de una ocasión actuaron como una especie de profesor particularmente poco paciente: primero rompían algo y después obligaban a entender por qué.

Medir antes de afirmar

La fase de optimización permitió comprobar además que las decisiones aparentemente pequeñas pueden producir mejoras relevantes cuando se acumulan.

El consumo de memoria en reposo se redujo de 985 MiB a 764 MiB, lo que supone una disminución aproximada de 221 MiB. De forma paralela, el tiempo de arranque pasó de 11,003 segundos a 8,223 segundos, una reducción cercana a los 2,8 segundos.

Estas mejoras no fueron consecuencia de reescribir el sistema ni de eliminar indiscriminadamente componentes. Surgieron principalmente de analizar qué servicios se ejecutaban, determinar cuáles necesitaban permanecer activos y modificar el comportamiento de aquellos que solo eran necesarios en situaciones concretas.

Ahí aparece otra de las conclusiones del desarrollo: optimizar no significa necesariamente quitar; significa utilizar cada recurso cuando corresponde.

Un sistema puede tener numerosas herramientas instaladas y seguir siendo razonablemente ligero si estas permanecen inactivas hasta que el usuario las necesita. Del mismo modo, un entorno aparentemente minimalista puede desperdiciar recursos si mantiene procesos innecesarios funcionando continuamente en segundo plano.

Las métricas resultaron especialmente importantes porque permitieron sustituir impresiones subjetivas por resultados medibles. Decir que una máquina “parece más rápida” tiene poco valor técnico. Poder comparar su consumo y sus tiempos antes y después de una modificación permite defender las decisiones con datos.

Una versión 0.1 que no pretende ser un punto final

Alcázar-OS v0.1 debe entenderse, por tanto, como una primera versión funcional y no como un producto terminado.

El sistema constituye una base sobre la que podrían desarrollarse versiones posteriores con un mayor nivel de automatización, nuevos flujos de trabajo, una integración más profunda entre Alcaza-IA y las herramientas disponibles, sistemas de generación aumentada mediante recuperación de información (RAG) o incluso una futura imagen ISO instalable que permita desplegar Alcázar-OS de forma más directa y reproducible.

También existe margen para investigar formas más avanzadas de conectar la inteligencia artificial local con los resultados generados por las diferentes herramientas del entorno. Sin embargo, cualquier evolución futura deberá conservar el principio que ha guiado buena parte del proyecto: añadir capacidades únicamente cuando exista una utilidad suficientemente clara para justificar su coste en complejidad, mantenimiento y recursos.

Porque un sistema especializado puede crecer hasta convertirse en algo muy potente o, con sorprendente facilidad, en un cajón lleno de herramientas que nadie recuerda por qué instaló.

Al final, probablemente el resultado más importante de Alcázar-OS no sea una determinada combinación de Debian, LXQt, Autopsy, Tor o un modelo Qwen. Es el proceso que permitió llegar hasta ella. Diseñar, probar, equivocarse, medir, corregir y documentar.

Ese ciclo ha transformado una idea inicial en un prototipo funcional que puede explicarse y defenderse tanto desde el punto de vista técnico como académico. Y quizá esa sea una de las diferencias esenciales entre ensamblar software y hacer ingeniería: en el primer caso importa que algo funcione; en el segundo, también importa comprender por qué funciona, qué compromisos se han aceptado para conseguirlo y qué ocurrirá cuando llegue el momento inevitable de cambiarlo.

Bibliografía

- [1] Debian Project, *Release Notes for Debian 13 (trixie)*. Debian Documentation Project, 2026. [En línea]. Disponible en: documentación oficial de Debian.
- [2] Debian Project, “Debian ‘trixie’ Release Information,” 2026. [En línea]. Disponible en: sitio oficial de Debian.
- [3] Debian Installer Team, *Debian trixie — Guía de instalación*. Debian Project, 2026. [En línea]. Disponible en: documentación oficial de Debian.
- [4] LXQt Project, “LXQt — The Lightweight Qt Desktop Environment,” 2026. [En línea]. Disponible en: sitio oficial de LXQt.
- [5] LXQt Project, “About LXQt,” 2026. [En línea]. Disponible en: documentación oficial de LXQt.
- [6] Xfce Development Team, “xfwm4 — Introduction,” *Xfce Documentation*, 2024. [En línea].
- [7] Xfce Development Team, “Thunar File Manager,” *Xfce Documentation*, 2025. [En línea].
- [8] SDDM Project, “SDDM: QML based X11 and Wayland Display Manager,” 2026. [En línea]. Disponible en: repositorio oficial de SDDM.
- [9] X.Org Foundation, *X.Org Developer Documentation*, 2026. [En línea].
- [10] Free Software Foundation, *GNU GRUB Manual 2.14*. GNU Project, 2026. [En línea].
- [11] P. Brown, J. Blandford, O. Taylor, V. Untz, W. Bastian, A. Lortie, D. Faure y W. Thompson, “Desktop Entry Specification,” ver. 1.5, freedesktop.org, 2020.
- [12] W. Bastian, F. Gouget, A. Graveley, G. Lebl, H. Pennington y H. Wendel, “Desktop Menu Specification,” ver. 1.1, freedesktop.org, 2016.
- [13] Oracle Corporation, *Oracle VirtualBox User Guide*, release 7.2, 2026.
- [14] Oracle Corporation, “Importing and Exporting Virtual Machines,” *Oracle VirtualBox 7.2 Documentation*, 2026.
- [15] Fastfetch Project, “Fastfetch: System Information Tool,” 2026. [En línea]. Disponible en: repositorio oficial del proyecto.
- [16] Jan, “Local AI Engine — llama.cpp,” *Jan Documentation*, 2026. [En línea].
- [17] Jan, “Model Management,” *Jan Documentation*, 2026. [En línea].
- [18] ggml-org, “llama.cpp: LLM inference in C/C++,” GitHub, 2026. [En línea].
- [19] ggml-org, “llama.cpp Quantization Documentation,” 2026. [En línea].
- [20] Qwen Team, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [21] E. Frantar, S. Ashkboos, T. Hoefler y D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” en *Proc. International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.17323.
- [22] J. Lin *et al.*, “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,” en *Proc. MLSys*, 2024, arXiv:2306.00978.

- [29] VirusTotal, *YARA Documentation*, ver. 4.5, 2026. [En línea].
- [30] SigmaHQ, *Sigma Specification — Generic Signature Format for SIEM Systems*, ver. 2.1, 2025. [En línea].
- [32] OASIS Open, *STIX Version 2.1*, OASIS Standard, 2021.
- [33] OASIS Open, *TAXII Version 2.1*, OASIS Standard, 2021.
- [34] MITRE, “MITRE ATT&CK — Enterprise Matrix,” 2026. [En línea].
- [35] National Institute of Standards and Technology, *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*, NIST SP 800-61 Rev. 3, 2025.
- [36] K. Kent, S. Chevalier, T. Grance y H. Dang, *Guide to Integrating Forensic Techniques into Incident Response*, NIST SP 800-86, 2006, doi: 10.6028/NIST.SP.800-86.
- [37] Human Rights Center, University of California Berkeley, y Office of the United Nations High Commissioner for Human Rights, *Berkeley Protocol on Digital Open Source Investigations*. United Nations, 2020.